

Curso Profesional de Entity Framework Core 8

Módulo 5 - Persistencia empresarial

TEORÍA

AUTOR: JAIME GALLO

.NET 8 · Entity Framework Core 8 · Visual Studio Community · SQL Server LocalDB · Proyecto AceriaData

Índice

Módulo 5 — Persistencia empresarial	4
Cómo utilizar este manual	4
Itinerario del módulo	4
Punto 5.1 — Concurrencia optimista: concepto y necesidad	4
Una orden, dos operadores y una decisión equivocada	4
Qué recuerda EF Core al leer una entidad	5
Experimento 1: se modifica la misma propiedad	5
Experimento 2: dos propiedades diferentes	6
Cómo identificar las propiedades realmente modificadas	6
Dos formas de coordinar escrituras	6
Diagnóstico de una incidencia real	6
Resultado esperado y evidencia concreta de los dos escenarios	6
Punto 5.2 — Configuración de tokens de concurrencia	7
Convertir una sospecha en una comprobación	7
Opción A: rowversion de SQL Server	7
Opción B: token de una propiedad de negocio	8
Qué NO proporciona automáticamente un token	8
Modelo, migración y base física: tres capas distintas	8
Laboratorio guiado: de IsRowVersion() a una excepción reproducible	8
Punto 5.3 — Resolución de conflictos de concurrencia	9
Detectar no es decidir	9
Escenario de referencia	9
Política 1: gana el cliente	10
Política 2: gana la base de datos	10
Política 3: fusión por propiedades	10
Política 4: notificar y pedir una decisión	10
Dos situaciones que requieren tratamiento especial	11
Implementar un reintento con límite sin ocultar un conflicto permanente	11
Punto 5.4 — Transacciones: SaveChanges y transacciones explícitas	11
La unidad de trabajo no siempre coincide con una instrucción	12
Caso A: una sola llamada a SaveChanges	12
Caso B: varias llamadas que deben confirmar juntas	12
Savepoints: revertir una parte sin perder toda la transacción	12
Savepoints automáticos y MARS	13
La ambigüedad de un fallo en Commit	13
Ejemplo de fallo inducido y verificación independiente	13
Punto 5.5 — Transacciones ambientales y buenas prácticas	14
Una operación atraviesa más de un componente	14
Construcción de un ámbito ambiental	14
Tres opciones que cambian el significado de un método	15
Dos DbContext no significan automáticamente dos transacciones distribuidas	15
Niveles de aislamiento: lo que prometen y lo que no	15
Una transacción de SQL Server no controla el mundo exterior	15
Experimento completo: dos contextos y una conexión física	16
Punto 5.6 — Migraciones en producción: estrategias y despliegue	16
Del comando de desarrollo al despliegue controlado	16
Cuatro estrategias y sus contextos	17
Construcción del artefacto antes del despliegue	17
El historial no es una tabla desechable	17
El preflight: comprobar antes de cambiar	18
Verificar que el esquema realmente cambió	18
Recuperación: por qué Down no equivale a backup	18
Aplicar migraciones mediante el servicio real IMigrator	18

Punto 5.7 – Migraciones idempotentes y scripts SQL	19
Una flota no siempre comparte la misma versión de esquema	19
Tres clases de scripts que no deben confundirse	19
Qué es una prueba de idempotencia convincente	19
Comprobar historia y esquema son pruebas distintas	20
Artefacto inmutable y versiones conocidas	20
Límites del concepto	20
Prueba más completa del mismo artefacto: historial y columna	20
Punto 5.8 – Migraciones en equipos: conflictos y buenas prácticas	21
El problema no es el nombre de un archivo	21
Reconstruir el punto común	21
Si la migración B todavía es privada	21
Cómo verificar la cadena fusionada	21
Si la migración ya se publicó	22
Qué revisar en un pull request	22
Dos laboratorios distintos: migraciones paralelas y ramas Git	22
Punto 5.9 – Repository y Unit of Work en aplicaciones empresariales	23
El problema arquitectónico que queremos resolver	23
Un repositorio que expresa intención	23
Por qué exponer IQueryable puede atravesar la frontera	24
Unit of Work: coordinar, no duplicar	24
La duración de DbContext forma parte del diseño	24
Qué puede demostrar un test con Moq	24
Un caso real de AceriaData, sin cambiar la cardinalidad	24
Punto 5.10 – Logging y diagnóstico de Entity Framework Core	25
Una incidencia: guardar una orden tarda más de lo esperado	25
Paso 1: empezar con ILogger estructurado	25
Paso 2: integrar Serilog sin perder la estructura	25
Paso 3: diferenciar SQL previsto del realmente ejecutado	26
Paso 4: DiagnosticListener como fuente de eventos internos	26
Paso 5: EventCounters de EF Core 8	27
Paso 6: Application Insights y Azure Monitor	27
Reconstruir la incidencia original	27
Secuencia de implementación real: ActivitySource, eventos, contadores y exportación	27
Punto 5.11 – Testing con EF Core	28
Elegir una prueba que pueda demostrar la afirmación	28
Primera capa: unit tests de Application con xUnit y Moq	28
Segunda capa: diferencias de proveedor	29
Tercera capa: integración real con migraciones	29
Prueba que sí exige rowversion real	30
Cuarta capa: HTTP end-to-end con WebApplicationFactory	30
Independencia, limpieza y confianza	31
Contraprueba real de proveedor: clave foránea InMemory frente a SQLite	31
Qué demuestra el test real de rowversion	32
Punto 5.12 – Buenas prácticas y anti-patrones de persistencia empresarial	32
Un anti-patrón debe tener un síntoma, no sólo un nombre	32
Caso 1: N+1 consultas al enumerar órdenes	32
Caso 2: over-fetching y estado rastreado innecesario	33
Caso 3: evaluación de cliente y traducción SQL	33
Caso 4: AsSplitQuery y consultas con múltiples colecciones	34
Caso 5: configuración, sargabilidad y abstracciones	34
Matriz final: síntoma, demostración y decisión	34
Cierre del módulo: un método común	35
Comparar realmente las tres implementaciones de N+1	35
Distinguir tres clases de evidencia	36
Glosario breve	36

Módulo 5 – Persistencia empresarial

.NET 8 • Entity Framework Core 8 • SQL Server LocalDB • Proyecto AceriaData

Cómo utilizar este manual

Los cuatro módulos anteriores han construido progresivamente **AceriaData**. El módulo 5 introduce una pregunta diferente: *¿qué ocurre cuando esa aplicación deja de ser un ejercicio aislado y debe resistir usuarios concurrentes, cambios de esquema, fallos parciales y despliegues reales?*

Una consulta que devuelve las filas correctas no demuestra, por sí sola, que una operación sea segura. Dos usuarios pueden leer simultáneamente una orden, modificarla y guardar con resultados distintos de los esperados; una transacción puede fallar después de enviar varias instrucciones; una migración puede funcionar en una máquina y dejar un entorno inconsistente. Tampoco basta con añadir una batería de tests: cada prueba debe usar un entorno capaz de reproducir el fenómeno que pretende verificar.

Cada punto comienza con una situación de negocio, explica el mecanismo de EF Core que interviene y ofrece una secuencia razonada de código, efectos esperados y decisiones. Los ejemplos se conectan con el proyecto acumulativo situado en M05/PROYECTO/5.1 ... M05/PROYECTO/5.12. Cuando se menciona SQL, se distingue entre **forma ilustrativa de la sentencia** y **SQL efectivamente observado**, que debe obtenerse al ejecutar el proyecto.

Itinerario del módulo

Punto	Pregunta que debemos poder responder
5.1	¿Cómo puede perderse una edición cuando dos usuarios trabajan sobre una misma fila?
5.2	¿Qué información necesita EF Core para detectar una edición obsoleta?
5.3	¿Qué debe hacer la aplicación cuando ya se detectó un conflicto?
5.4	¿Qué operaciones se confirman o revierten juntas?
5.5	¿Qué límites tiene una transacción que atraviesa varios componentes?
5.6	¿Cómo se entrega un cambio de esquema de manera controlada?
5.7	¿Qué garantiza y qué no garantiza un script idempotente?
5.8	¿Cómo se integran migraciones creadas por desarrolladores distintos?
5.9	¿Cuándo aporta valor una capa Repository/Unit of Work sobre EF Core?
5.10	¿Cómo observamos lo que hace EF Core sin confundir logs, eventos y métricas?
5.11	¿Qué debe probarse con mocks y qué exige SQL Server real?
5.12	¿Cómo reconocemos y corregimos anti-patrones con evidencia?

Punto 5.1 – Concurrencia optimista: concepto y necesidad

Una orden, dos operadores y una decisión equivocada

Un cliente comunica una corrección en su orden de fabricación. La operadora **Ana** abre el pedido y empieza a modificar el nombre del cliente. Casi al mismo tiempo, **Bruno** abre la misma orden desde otra pantalla. Ambos han leído el estado inicial. Ana guarda su corrección; unos segundos después Bruno guarda otro nombre de cliente sin conocer el cambio anterior. El resultado no es un fallo de SQL Server: las dos sentencias pueden ejecutarse correctamente. Es un fallo en la **semántica de la operación**: Bruno ha escrito tomando como válida información que ya había quedado obsoleta.

Llamaremos **actualización perdida** al caso en el que una escritura posterior reemplaza un cambio confirmado

anteriormente, sin advertir al segundo actor. Para reproducirlo necesitamos tres elementos: dos lecturas previas de la misma fila, una modificación de la misma propiedad en ambas copias y dos escrituras sucesivas.

Es esencial que las copias correspondan a **dos DbContext independientes**. Una misma instancia de DbContext no representa dos peticiones separadas: mantiene una identidad de entidad y un ChangeTracker común. AceriaData usa scopes distintos del contenedor de dependencias para reproducir el escenario.

Qué recuerda EF Core al leer una entidad

Cuando consultamos una entidad con tracking, EF Core conserva un conjunto de valores originales y observa los actuales. Al invocar SaveChanges, detecta las propiedades modificadas y construye las operaciones de base de datos correspondientes. Sin una propiedad configurada como token de concurrencia, la cláusula de actualización puede identificar la fila por su clave primaria sin comprobar que otro actor la haya modificado entre la lectura y la escritura.

Una simplificación frecuente es afirmar que EF Core siempre reescribe todas las columnas. **No es correcto** para una entidad rastreada de manera normal: EF Core puede limitar el UPDATE a las propiedades marcadas como modificadas. El comportamiento puede cambiar cuando se reciben objetos desconectados y se utiliza Update marcando la entidad completa como modificada. Por tanto, debemos distinguir dos problemas: detectar que *la fila ha cambiado* y determinar si *se ha perdido una modificación concreta*.

Experimento 1: se modifica la misma propiedad

El siguiente fragmento muestra el núcleo del experimento con el modelo de AceriaData. Se inserta en un contexto donde ya existen IServiceScopeFactory, el número de una orden válida y los servicios de Infrastructure configurados. En el proyecto real, el caso está encapsulado en ConcurrenciaOptimistaM5UseCase.

```
// Dos scopes => dos unidades de trabajo y dos DbContext independientes.
using var scopeA = scopeFactory.CreateScope();
using var scopeB = scopeFactory.CreateScope();
var dbA = scopeA.ServiceProvider.GetRequiredService<AceriaDbContext>();
var dbB = scopeB.ServiceProvider.GetRequiredService<AceriaDbContext>();

// Ambos leen ANTES de que se produzca ninguna escritura.
var a = dbA.OrdenesFabricacion.Single(o => o.NumeroOrden == numeroOrden);
var b = dbB.OrdenesFabricacion.Single(o => o.NumeroOrden == numeroOrden);

// A confirma su cambio.
a.Cliente = "Cliente corregido por Ana";
dbA.SaveChanges();

// B aún conserva la copia anterior de la misma fila.
b.Cliente = "Cliente escrito por Bruno";
dbB.SaveChanges();

// Verificación independiente del estado realmente persistido.
using var verificacion = scopeFactory.CreateScope();
var dbVerificacion = verificacion.ServiceProvider
    .GetRequiredService<AceriaDbContext>();
var resultado = dbVerificacion.OrdenesFabricacion
    .AsNoTracking()
    .Single(o => o.NumeroOrden == numeroOrden);
Console.WriteLine(resultado.Cliente);
```

Lectura paso a paso. En las dos primeras consultas se materializan dos objetos diferentes con los mismos valores iniciales. dbA.SaveChanges() confirma la intención de Ana. Cuando Bruno guarda, la clave primaria sigue identificando una fila válida: sin un token que verifique la versión original, EF Core no tiene por qué detectar que el nombre cambió entre medias. En este escenario, el último valor puede prevalecer. La última lectura usa un tercer contexto y AsNoTracking() para no reutilizar ninguna instancia previamente rastreada.

Qué observaríamos: el valor final de Cliente; las columnas realmente incluidas en cada UPDATE; las filas afectadas; y si se lanzó una excepción. La sentencia que interesa tiene, esquemáticamente, una forma como UPDATE ... SET Cliente = ... WHERE Id = Esta línea es **pseudocódigo SQL explicativo**, no una captura real. Para comprobar la sentencia generada hay que consultar el logging o un interceptor de comandos: ToQueryString() no está diseñado para capturar directamente el SQL ejecutado por SaveChanges.

Experimento 2: dos propiedades diferentes

Repetimos el experimento desde un estado conocido, pero esta vez Ana cambia Cliente y Bruno cambia Estado. Si ambos usan entidades cargadas normalmente y sólo se marca como modificada la propiedad correspondiente, las dos actualizaciones pueden coexistir sin que el valor de Cliente sea sustituido al guardar Estado.

// Asumimos dos contextos con copias ya cargadas del mismo registro.

```
a.Cliente = "Cliente corregido";
dbA.SaveChanges();
```

```
b.Estado = "EnRevision";
dbB.SaveChanges();
```

// Consultar con un tercer contexto qué valores han quedado.

El resultado combinado puede ser técnicamente correcto y, aun así, contravenir una regla de negocio. Por ejemplo, quizá no sea válido cambiar el estado de una orden sin volver a revisar determinados datos. La ausencia de actualización perdida en columnas distintas **no equivale** a garantizar consistencia de negocio. Esta distinción justifica que el mecanismo de concurrencia y las validaciones de dominio sean complementarios.

Cómo identificar las propiedades realmente modificadas

Antes de llamar a SaveChanges, es posible inspeccionar el estado de EF Core:

```
var entry = dbB.Entry(b);
dbB.ChangeTracker.DetectChanges(); // Necesario si queremos inspección explícita y fiable.
foreach (var propiedad in entry.Properties)
{
    Console.WriteLine($"{propiedad.Metadata.Name}: {propiedad.IsModified}");
}
```

DetectChanges() no es la solución a un conflicto de concurrencia: fuerza la detección de cambios locales cuando la inspección lo requiere. Si el problema es que otra conexión modificó la fila, necesitamos **comparar una versión original con la versión actual de base**, mecanismo que incorporaremos en 5.2.

Dos formas de coordinar escrituras

La **concurrencia optimista** permite leer y editar sin mantener una transacción y un bloqueo durante toda la interacción humana. Cuando llega el momento de guardar, verifica si la versión leída sigue siendo válida y comunica un conflicto cuando no lo es. Es adecuada, especialmente, cuando los choques no son la operación habitual y existe una política razonable de resolución.

La **coordinación pesimista** emplea mecanismos de bloqueo o instrucciones transaccionales para impedir determinadas interferencias antes o durante la escritura. Puede ser apropiada en operaciones muy breves y altamente contenciosas, pero mantener bloqueos mientras alguien edita una pantalla incrementa costes y riesgos. Ninguna estrategia constituye por sí sola una solución universal.

Diagnóstico de una incidencia real

Cuando un usuario afirma que «EF ha pisado mis datos», la investigación debe reconstruir **quién leyó qué valor, cuándo y con qué DbContext**, qué propiedades estaban marcadas como modificadas y qué sentencias llegaron realmente al servidor. Debe revisarse también si la entidad fue cargada y rastreada o si procedía de un DTO desconectado al que se aplicó Update. De otro modo es fácil atribuir a concurrencia un problema de estado desconectado o de una actualización demasiado amplia.

Conclusión del punto. Hemos demostrado el problema sin introducir aún una protección. La próxima pregunta ya es concreta: ¿qué dato adicional necesita incluir EF Core en la condición de escritura para saber que estamos intentando guardar sobre una versión antigua?

Resultado esperado y evidencia concreta de los dos escenarios

Situación	Escritura A	Escritura B	Resultado sin token	Qué inspeccionar
Ambos cambian Cliente	Cliente = "A"	Cliente = "B"	Predomina B si B guarda el último y no hay otro control.	SQL de los dos UPDATE; estado final del tercer contexto.

Situación	Escritura A	Escritura B	Resultado sin token	Qué inspeccionar
A cambia Cliente; B cambia Estado	Cliente = "A"	Estado = "EnRevision"	Pueden conservarse ambas columnas con tracking habitual.	Conjunto de propiedades IsModified, columnas del SET.
B reconecta un objeto y lo marca completamente Modified	Cliente = "A"	Todas las propiedades consideradas modificadas	B puede sobrescribir valores que no pretendía editar.	Entry.State, SQL y datos enviados desde el cliente.

Esta tabla describe el **comportamiento esperado bajo las condiciones indicadas**. Para verificarlo, la prueba debe fijar el conjunto de datos, leer ambas copias antes de escribir, contar comandos con el interceptor y hacer la consulta final con un contexto distinto. Conviene registrar asimismo el valor original para distinguir una pérdida real de una sobrescritura voluntaria.

Punto 5.2 – Configuración de tokens de concurrencia

Convertir una sospecha en una comprobación

En 5.1 vimos que un segundo SaveChanges puede aceptar una modificación basada en una copia obsoleta. El objetivo ahora no es impedir que dos usuarios lean una orden, sino conseguir que **el segundo guardado detecte la incompatibilidad**. Para ello EF Core permite configurar una o varias propiedades como **tokens de concurrencia**: conserva su valor original y lo añade a la condición de actualización o eliminación.

El cambio conceptual es sencillo, aunque sus consecuencias son importantes. Antes, una actualización podía depender sólo del identificador de fila; ahora depende del identificador y de un valor observado al leer. Si ese valor ha cambiado, la operación que esperaba afectar una fila no encuentra ninguna que cumpla la condición. EF Core comunica entonces un `DbUpdateConcurrencyException`.

Opción A: rowversion de SQL Server

En SQL Server, rowversion es un valor binario generado por el motor que cambia cuando se actualiza una fila. No almacena una fecha ni representa una hora. Tampoco es un índice. Su utilidad aquí es actuar como señal de que el estado de la fila ha avanzado desde la lectura.

En una entidad suele representarse con una propiedad `byte[]`. La configuración es explícita:

```
// Fragmento de configuración de la entidad OrdenFabricacion.
modelBuilder.Entity<OrdenFabricacion>()
    .Property(o => o.RowVersion)
    .IsRowVersion();
```

`IsRowVersion()` configura la propiedad como token de concurrencia y como valor generado por la base durante inserciones y actualizaciones. El nombre de la propiedad, por sí solo, no debe sustituir a la comprobación de la configuración efectiva. En el proyecto, la declaración se integra en las configuraciones de Infraestructure y el cambio se materializa mediante la migración **M5_5_2_ConcurrencyTokens**.

Ciclo de una actualización protegida

1. Ana y Bruno leen la misma orden y reciben el mismo RowVersion original.
2. Ana cambia Cliente y guarda; SQL Server actualiza la fila y genera otro valor de rowversion.
3. Bruno intenta guardar con el token original que leyó anteriormente.
4. La condición de la segunda actualización ya no coincide con la fila actual.
5. EF Core observa que la operación esperada no actualizó ninguna fila y lanza `DbUpdateConcurrencyException`.

La lógica puede visualizarse con esta **representación SQL simplificada**:

```
-- Ilustración del mecanismo; NO es SQL capturado del proveedor.
UPDATE dbo.OrdenesFabricacion
SET Cliente = @nuevoCliente
WHERE Id = @id AND RowVersion = @versionQueLeiElUsuario;
```

La sentencia real puede incorporar distintas formas de devolver valores generados y no debe suponerse idéntica a este esquema. En un diagnóstico es preferible observar el comando ejecutado mediante logs o `DbCommandInterceptor`.

Opción B: token de una propiedad de negocio

No siempre queremos considerar conflictiva **cualquier** edición de la fila. Supongamos que la decisión operativa depende exclusivamente del estado de un detalle. Podemos configurar EstadoDetalle como token:

// Fragmento sobre la entidad DetalleOrden.

```
modelBuilder.Entity<DetalleOrden>()
    .Property(d => d.EstadoDetalle)
    .IsRequired()
    .HasMaxLength(50)
    .HasDefaultValue("Pendiente")
    .IsConcurrencyToken();
```

Si otro usuario cambia EstadoDetalle, el valor original deja de coincidir y se detecta el conflicto. Si cambia una propiedad distinta pero EstadoDetalle permanece igual, ese token **no** representa un cambio de versión para la propiedad elegida. Por eso la elección del token expresa una regla de negocio: qué modificaciones vuelven obsoleta una decisión.

También pueden utilizarse tokens gestionados por la aplicación, por ejemplo un Guid, siempre que el código actualice su valor conforme a la política deseada. Esta técnica reduce dependencia del proveedor, pero exige disciplina: si la aplicación olvida renovar el token, no se detectarán todos los conflictos pretendidos.

Qué NO proporciona automáticamente un token

Una columna rowversion no crea un índice específico. EF Core utiliza la columna como parte del predicado de escritura, pero **configuración de concurrencia** e **indexación física** son decisiones independientes. Para conocer el estado real de SQL Server se examinan `sys.columns`, `sys.indexes` y `sys.index_columns`; no debe inferirse que existe un índice sólo por el nombre o tipo de la columna.

Tampoco proporciona una política para resolver conflictos: únicamente los **detecta**. Si la excepción se captura y se vuelve a intentar exactamente el mismo guardado sin actualizar la referencia de versión, el problema seguirá existiendo.

Modelo, migración y base física: tres capas distintas

Añadir RowVersion a C# no transforma automáticamente una base ya desplegada. Para que el mecanismo funcione de extremo a extremo deben alinearse el **modelo de EF Core**, el **snapshot y migración**, y el **esquema físico** al que se conecta la aplicación. En AceriaData, el punto 5.2 introduce el cambio de modelo oficial; los puntos posteriores mantienen esa cadena sin añadir migraciones ficticias.

Para investigar un fallo de concurrencia que no aparece, verificamos por este orden: si la propiedad está reconocida como token en el modelo; si la migración aplicada contiene la columna esperada; si la conexión apunta a la base correcta; si el comando de escritura incluye el token original; y si los dos actores cargaron sus entidades **antes** de la primera escritura. Sin estas condiciones es posible ejecutar un test que no reproduce el conflicto.

Conclusión del punto. rowversion permite detectar cualquier actualización de la fila en SQL Server; un token de propiedad puede restringir el conflicto a determinados cambios. La excepción que generan no responde todavía a la pregunta que hará un usuario: «¿y ahora qué ocurre con mis datos?».

Laboratorio guiado: de IsRowVersion() a una excepción reproducible

El primer enlace verificable se encuentra en ConcurrencyTokensConfiguration.cs (carpeta Persistence/Configuratio de Infrastructure) del punto 5.2. El proyecto configura RowVersion mediante IsRowVersion() para **OrdenFabricacion**, **PlanchaAcero** y **Aleacion**, mientras DetalleOrden.EstadoDetalle emplea IsConcurrencyToken() y un valor por defecto. Son **dos mecanismos distintos**: SQL Server genera los bytes de rowversion automáticamente, pero no modifica por arte de magia el valor de un token de propiedad mantenido por el negocio.

El segundo enlace es la migración real 20260930203405_M5_5_2_ConcurrencyTokens.cs, que incorpora tres columnas rowversion y EstadoDetalle a la cadena de migraciones. El tercer enlace es la base física que usa el alumno: GetAppliedMigrations() debe contener M5_5_2_ConcurrencyTokens, y el esquema debe exponer RowVersion con el tipo del proveedor.

// Consultar el modelo efectivo (fragmento para un método con DbContext).

```
var entidad = context.Model.FindEntityType(typeof(OrdenFabricacion));
var propiedad = entidad.FindProperty(nameof(OrdenFabricacion.RowVersion));
Console.WriteLine($"Token: {propiedad.IsConcurrencyToken}");
Console.WriteLine($"Generación: {propiedad.ValueGenerated}");
```

// Consultar migraciones realmente aplicadas, no nombres supuestos.

```
var aplicadas = context.Database.GetAppliedMigrations();
Console.WriteLine(aplicadas.LastOrDefault());
```

El cuarto enlace es la escritura: **A y B deben consultar antes de que A escriba**; después A modifica y guarda, B modifica su copia antigua y `SaveChanges()` de B debe lanzar `DbUpdateConcurrencyException`. El guardado utiliza el valor original de la versión; no basta con comparar las propiedades en memoria o asignar manualmente un nuevo `byte[]`.

Verificación	Evidencia esperada	Error habitual si falta
Metadata	<code>IsConcurrencyToken == true</code> ; generación de versión del proveedor	Propiedad CLR sin configuración efectiva.
Migración	Identificador real en el historial de migraciones	Modelo nuevo frente a base antigua.
Esquema	Columna <code>rowversion</code> de SQL Server	Se está usando otra base o proveedor.
Dos contextos	Dos lecturas previas y segunda escritura rechazada	B lee <i>después</i> de A y no tiene copia obsoleta.
Índice	Inspección independiente de <code>sys.indexes</code>	Suponer que <code>rowversion</code> implica índice automático.

Sobre el SQL. Un predicado esquemático sería `WHERE Id = @id AND RowVersion = @versionOriginal`. La sentencia concreta, parámetros y comprobación de filas afectadas deben observarse con logging/interceptor sobre SQL Server; `ToQueryString()` no es evidencia de la ejecución de `SaveChanges()`.

Punto 5.3 – Resolución de conflictos de concurrencia

Detectar no es decidir

Cuando EF Core lanza `DbUpdateConcurrencyException`, la aplicación sabe que la escritura no cumplió las condiciones de versión. **No sabe automáticamente cuál de las dos ediciones tiene razón.** En una orden de fabricación, conservar un estado avanzado por el supervisor puede ser obligatorio, mientras que una corrección ortográfica del cliente quizá pueda recuperarse sin perjudicar el proceso.

Por eso es peligroso sustituir toda la explicación por `catch (DbUpdateConcurrencyException) { SaveChanges(); }`. La excepción debe convertirse en una decisión explícita. Para entender las opciones distinguimos tres conjuntos de valores:

Conjunto	Qué representa
<code>OriginalValues</code>	Los valores de referencia que EF Core conserva para la comprobación de concurrencia.
<code>CurrentValues</code>	La intención de escritura que mantiene el contexto local.
<code>GetDatabaseValues()</code>	Los valores que existen ahora en la fila de base de datos. Puede devolver <code>null</code> si ya no existe.

Escenario de referencia

Ana y Bruno leen una orden con `Cliente = "Acería Norte"` y `Estado = "Pendiente"`. Ana cambia el estado a `EnFabricacion` y guarda. Bruno, que conserva la copia anterior, corrige el cliente a `Acería del Norte` y trata de guardar. La política de negocio podría permitir conservar **el cliente de Bruno y el estado confirmado por Ana**, siempre que las reglas del dominio lo autoricen.

Una representación de la intención final sería:

Propiedad	Original	Bruno intenta	Base actual	Fusión deseada
Cliente	Acería Norte	Acería del Norte	Acería Norte	Acería del Norte
Estado	Pendiente	Pendiente	EnFabricacion	EnFabricacion

Esta tabla es un **escenario didáctico**, no un volcado de una ejecución.

Política 1: gana el cliente

La aplicación reconoce el conflicto, obtiene el estado actual de la base, acepta expresamente la intención local y actualiza la referencia original para reintentar. Esta elección puede sobrescribir cambios ya confirmados por otra persona, de manera que debería reservarse a casos cuya semántica lo permita.

```
// Fragmento para usar dentro del tratamiento de una excepción de concurrencia.
catch (DbUpdateConcurrencyException ex)
{
    var entrada = ex.Entries.Single();
    var baseActual = entrada.GetDatabaseValues();
    if (baseActual is null)
        throw new InvalidOperationException("La orden ha sido eliminada.");

    // Conservar CurrentValues; actualizar sólo la referencia de versión.
    entrada.OriginalValues.SetValues(baseActual);
    db.SaveChanges(); // Reintento de la intención local: requiere límite externo.
}
```

El fragmento ilustra un **único reintento**. En código real deben definirse el límite, el tratamiento de nuevas excepciones y la política de negocio. Actualizar `OriginalValues` no significa desactivar la concurrencia; significa reintentar contra una versión ya conocida.

Política 2: gana la base de datos

Si los cambios del servidor tienen prioridad, podemos descartar la intención local y recargar la entidad:

```
catch (DbUpdateConcurrencyException ex)
{
    foreach (var entrada in ex.Entries)
        entrada.Reload(); // Sustituye valores originales y actuales desde la base.
}
```

Esto puede resolver el conflicto técnico, pero **pierde los cambios locales**. Si el usuario llevaba veinte minutos editando, esa pérdida debe comunicarse de forma adecuada. En una interfaz profesional es frecuente ofrecer una comparación antes de descartar la edición.

Política 3: fusión por propiedades

La aplicación puede decidir que Cliente prevalezca desde el contexto local mientras Estado se tome de la base. Primero debe obtenerse una referencia actual, después construir conscientemente los valores que se van a guardar, y finalmente volver a validar las invariantes del dominio.

```
catch (DbUpdateConcurrencyException ex)
{
    var entrada = ex.Entries.Single();
    var baseActual = entrada.GetDatabaseValues();
    if (baseActual is null)
        throw new InvalidOperationException("La orden ya no existe.");

    // La edición del cliente se conserva en CurrentValues.
    // El estado confirmado por otro actor tiene prioridad.
    entrada.CurrentValues[nameof(OrdenFabricacion.Estado)] =
        baseActual[nameof(OrdenFabricacion.Estado)];

    // El próximo WHERE debe partir de la versión que existe ahora.
    entrada.OriginalValues.SetValues(baseActual);
    db.SaveChanges(); // Reintento controlado, sujeto a validación del dominio.
}
```

La fusión es especialmente delicada en propiedades relacionadas: cantidades, precios, estados de workflow o totales calculados no siempre pueden combinarse de manera independiente. Un resultado técnicamente guardable no es necesariamente válido para el negocio.

Política 4: notificar y pedir una decisión

En lugar de reintentar, una API puede devolver una representación del conflicto que incluya el valor originalmente leído, el valor que el usuario pretendía guardar y el valor actual del servidor. La interfaz puede mostrar la diferencia y solicitar una decisión informada. Esta opción introduce trabajo de UX, pero es la más transparente cuando perder

una edición puede tener consecuencias relevantes.

Dos situaciones que requieren tratamiento especial

La fila ya no existe. Si `GetDatabaseValues()` devuelve `null`, no tenemos una versión nueva contra la que fusionar. El recurso fue eliminado o ya no es accesible. La aplicación puede informar, cancelar o proponer una recreación cuando tenga sentido, pero no debe seguir reintentando una actualización imposible.

El conflicto se repite. Incluso después de renovar la referencia, un tercer actor puede modificar la fila antes de que guardemos. Un bucle de reintento sin límite puede consumir recursos indefinidamente. Definimos un máximo de intentos, registramos cada conflicto y abandonamos o solicitamos intervención cuando se supera el umbral. La operación completa debe ser segura de repetir: si cada intento produce efectos externos adicionales, reintentar podría duplicarlos.

Conclusión del punto. Resolver un conflicto requiere separar el mecanismo técnico de detección de una **política de negocio**. La elección correcta depende de qué información puede prevalecer y de si el usuario acepta la pérdida, fusión o repetición de su edición.

Implementar un reintento con límite sin ocultar un conflicto permanente

La política cliente-gana puede repetirse sólo si el caso de uso lo admite. El siguiente fragmento muestra **dónde se decide renovar `OriginalValues` y cuándo abandonar**, suponiendo un contexto `db` que ya rastrea una entidad modificada; no sustituye el resto de la lógica de validación del dominio.

```
const int maxIntentos = 3;
bool guardado = false;
for (int intento = 1; intento <= maxIntentos; intento++)
{
    try
    {
        db.SaveChanges();
        guardado = true;
        break;
    }
    catch (DbUpdateConcurrencyException ex)
    {
        if (intento == maxIntentos) throw; // No reintento infinito.
        foreach (var entrada in ex.Entries)
        {
            var baseActual = entrada.GetDatabaseValues();
            if (baseActual is null)
                throw new InvalidOperationException("Fila eliminada.");
            entrada.OriginalValues.SetValues(baseActual);
        }
    }
}
Console.WriteLine($"Guardado: {guardado}");
```

En cada conflicto `GetDatabaseValues()` obtiene una versión más reciente; al reintentar, la aplicación acepta que los valores que permanecen en `CurrentValues` puedan prevalecer. Esto **no** constituye un merge selectivo y podría sobrescribir cambios de otro actor. Para un merge hay que ajustar explícitamente `CurrentValues`, validar invariantes y sólo entonces continuar.

Comprobación del resultado: un test debe comprobar no sólo que `guardado` sea verdadero, sino también el estado final de `Cliente`, `Estado` y el número de intentos. `AceriaData` ya separa los métodos `ClienteGana`, `BaseDeDatosGana`, `ResolucionPersonalizada`, `NotificarSinSobrescribir`, `ReintentoAcotado` y `DetectarFilaEliminada` en el repositorio del punto 5.3. La secuencia de aprendizaje conecta cada política con esos métodos del proyecto.

Punto 5.4 — Transacciones: `SaveChanges` y transacciones explícitas

La unidad de trabajo no siempre coincide con una instrucción

Una orden de fabricación puede requerir crear una cabecera, registrar detalles y reservar material. Si la cabecera se confirma pero falla el último paso, la base queda en un estado parcial que quizá el negocio no admite. Necesitamos definir **qué conjunto de cambios debe confirmarse entero o no confirmarse**: ésa es la frontera de atomicidad.

Una llamada a `SaveChanges` sobre un proveedor relacional como SQL Server ejecuta de forma transaccional su conjunto de cambios cuando es necesario. Eso permite insertar varias entidades relacionadas y evitar que sólo una de ellas quede confirmada cuando falla otra operación de la misma llamada. No significa, sin embargo, que varias llamadas independientes a `SaveChanges` queden unidas automáticamente.

Caso A: una sola llamada a `SaveChanges`

// Fragmento: CrearOrden y CrearDetalle representan constructores del dominio.

```
var orden = CrearOrden();
var detalle = CrearDetalle(orden);
db.OrdenesFabricacion.Add(orden);
db.DetallesOrden.Add(detalle);
db.SaveChanges();
```

Si las dos escrituras forman parte de la misma operación y una restricción de base impide guardar el detalle, se espera que la transacción de ese `SaveChanges` impida la confirmación parcial. Para **demostrarlo**, no basta con capturar la excepción: hay que consultar con otro contexto que no exista la cabecera recién insertada. En el proyecto real puede ser necesario preparar datos que provoquen una violación de clave o restricción; el fragmento anterior no genera por sí solo ese error.

Caso B: varias llamadas que deben confirmar juntas

Hay procesos que deben conocer una clave generada por la primera escritura antes de preparar el resto, o que combinan cambios EF Core con comandos sobre la misma conexión. Una transacción explícita permite realizar varias llamadas y posponer la confirmación global:

```
await using var transaccion = await db.Database.BeginTransactionAsync();
try
{
    db.OrdenesFabricacion.Add(orden);
    await db.SaveChangesAsync();

    db.DetallesOrden.Add(detalle);
    await db.SaveChangesAsync();

    await transaccion.CommitAsync();
}
catch
{
    await transaccion.RollbackAsync();
    throw;
}
```

Qué cambia respecto al caso A: la primera llamada a `SaveChangesAsync()` envía y procesa su trabajo, pero no confirma por sí sola la transacción exterior. Si falla la segunda etapa y se revierte la transacción, la primera tampoco debe quedar confirmada en SQL Server. Eso sí: los objetos .NET y el `ChangeTracker` **no retroceden mágicamente** al estado anterior. Puede ser necesario descartar el contexto, limpiar el tracking o recargar datos antes de continuar.

Mantener una transacción abierta mientras se ejecuta una llamada HTTP, se espera una respuesta humana o se realiza un cálculo largo puede aumentar la contención. Es preferible fijar fronteras cortas y explícitas.

Savepoints: revertir una parte sin perder toda la transacción

Un **savepoint** marca una posición intermedia dentro de la misma transacción. Permite deshacer escrituras posteriores a esa marca sin abandonar necesariamente los cambios anteriores. Veamos una transacción que admite que la segunda operación sea opcional:

```
await using var tx = await db.Database.BeginTransactionAsync();

db.OrdenesFabricacion.Add(primerOrden);
await db.SaveChangesAsync();

await tx.CreateSavepointAsync("AntesSegundaOrden");

db.OrdenesFabricacion.Add(segundaOrden);
await db.SaveChangesAsync();

// Deshacer sólo las modificaciones posteriores al savepoint.
await tx.RollbackToSavepointAsync("AntesSegundaOrden");
await tx.CommitAsync();
```

El estado final esperado de la base es: primera orden confirmada y segunda orden ausente. Para evitar que el ChangeTracker induzca conclusiones equivocadas, la verificación debe abrir otro contexto o renovar explícitamente su estado. En SQL Server, los savepoints se corresponden conceptualmente con SAVE TRANSACTION y ROLLBACK TRANSACTION; no debemos exigir una sintaxis RELEASE SAVEPOINT propia de otros motores.

Savepoints automáticos y MARS

Cuando SaveChanges se llama dentro de una transacción que ya existe, EF Core puede crear automáticamente un savepoint antes de guardar. De ese modo, ante determinados errores puede devolver la transacción al estado anterior a ese SaveChanges y permitir un tratamiento controlado.

Limitación importante: en SQL Server, los savepoints automáticos de EF Core no son compatibles con **Multiple Active Result Sets (MARS)** habilitado. Cuando el curso pretende demostrar estos mecanismos, la conexión se configura con MultipleActiveResultSets=false. No basta con que no estemos usando MARS activamente: la configuración de la conexión es relevante.

La ambigüedad de un fallo en Commit

Un error de Commit merece un tratamiento más cuidadoso que «si falla, todo queda revertido». Si se interrumpe la conexión durante la confirmación, desde el cliente puede ser difícil saber si el servidor llegó a confirmar la transacción. Reintentar sin verificar puede duplicar una operación no idempotente. Los mecanismos de resiliencia deben conocer esta posibilidad y, cuando corresponda, usar una identidad estable de operación o una verificación posterior.

Tampoco un Rollback puede deshacer un correo ya enviado o una solicitud HTTP que terminó satisfactoriamente. La transacción controla los recursos que participan en ella; la recuperación completa del negocio es una cuestión mayor que estudiaremos en 5.5.

Conclusión del punto. Utilizamos una única llamada a SaveChanges cuando esa llamada expresa la unidad atómica; una transacción explícita cuando debemos coordinar varias escrituras; y savepoints cuando tiene sentido recuperar una etapa intermedia. La decisión parte de la **semántica del negocio**, no de añadir transacciones por costumbre.

Ejemplo de fallo inducido y verificación independiente

El punto 5.4 no termina cuando se observa una excepción: termina al demostrar qué datos **permanecen** después. Para una única llamada a SaveChanges, introducir una entidad válida y otra que viole una restricción en la **misma unidad** permite comprobar atomicidad. Al inspeccionar el resultado desde otro contexto, la entidad válida no debe aparecer como confirmada de forma aislada si falla la llamada completa.

Con una transacción explícita, la comprobación es diferente: la primera llamada puede llegar a SQL Server sin confirmar; la segunda falla, el código ejecuta Rollback, y una lectura independiente confirma que tampoco persistió el primer cambio. La distinción entre “se envió el comando” y “quedó confirmado” es esencial.

```

await using var tx = await db.Database.BeginTransactionAsync();
try
{
    db.OrdenesFabricacion.Add(ordenValida); // Preparadas antes.
    await db.SaveChangesAsync();
    db.OrdenesFabricacion.Add(ordenInvalida); // Provoca fallo controlado.
    await db.SaveChangesAsync();
    await tx.CommitAsync();
}
catch
{
    await tx.RollbackAsync();
    db.ChangeTracker.Clear(); // La memoria no retrocede automáticamente.
    throw;
}
// Reabrir otro DbContext para comprobar lo persistido.

```

ordenValida y ordenInvalida son objetos preparados por el laboratorio y el segundo **debe infringir una restricción real** del esquema. Por ello es un fragmento de razonamiento, no una prueba autocontenida. Hay que revisar también qué hace el tracker tras volver a un savepoint: una reversión de SQL no restaura automáticamente las propiedades CLR y estados Added/Unchanged.

MARS. Los savepoints automáticos de EF Core bajo una transacción existente no se crean cuando SQL Server tiene habilitado Multiple Active Result Sets. El laboratorio usa `MultipleActiveResultSets=false`; no es un requisito de toda aplicación SQL Server, sino la condición necesaria para confiar en ese comportamiento específico.

Punto 5.5 – Transacciones ambientales y buenas prácticas

Una operación atraviesa más de un componente

Imaginemos un servicio que crea una orden mediante un repositorio y registra la primera plancha mediante otro. Si ambos utilizan el mismo DbContext, conocemos la frontera de escritura: podemos agrupar los cambios en SaveChanges o una transacción explícita. Pero ¿qué sucede cuando los componentes crean contextos diferentes y ningún método recibe un objeto DbTransaction?

.NET proporciona TransactionScope, que permite definir una **transacción ambiental**. Mientras se ejecuta el bloque, los recursos compatibles pueden descubrir la transacción a través de Transaction.Current y participar en ella. Es una herramienta de composición; también añade un contrato implícito que debe hacerse visible en la arquitectura.

Construcción de un ámbito ambiental

```

var opciones = new TransactionOptions
{
    IsolationLevel = System.Transactions.IsolationLevel.ReadCommitted,
    Timeout = TimeSpan.FromSeconds(30)
};

using var scope = new TransactionScope(
    TransactionScopeOption.Required,
    opciones,
    TransactionScopeAsyncFlowOption.Enabled);

// Dentro del scope, Transaction.Current representa el ambiente.
Console.WriteLine(Transaction.Current is not null);
await RealizarTrabajoDeBaseDeDatosAsync();

```

```

// Complete expresa la intención de confirmar la unidad.
scope.Complete();

```

TransactionScope no se confirma por llamar a SaveChanges: normalmente es Complete() quien indica que el trabajo del ámbito se ha realizado satisfactoriamente. Si se abandona el scope sin esa llamada, la transacción debe abortarse según la semántica del recurso participante. Si una excepción ocurre después de Complete() o durante la disposición, el tratamiento debe considerar el estado efectivo de los recursos.

La opción TransactionScopeAsyncFlowOption.Enabled es imprescindible para expresar correctamente la

propagación ambiental a través de `await` en el flujo utilizado. Sin ella, el comportamiento de las continuaciones puede generar errores o no preservar el contexto transaccional pretendido.

Tres opciones que cambian el significado de un método

Opción	Qué ocurre si existe una transacción ambiental
Required	El bloque participa en la transacción existente. Si no la hay, crea una.
RequiresNew	Crea una nueva frontera transaccional independiente del ámbito exterior.
Suppress	Ejecuta el bloque suprimiendo la transacción ambiental.

La distinción importa en casos reales. Supongamos un método principal que registra una orden y otro método que añade una observación. Si el segundo usa `Required`, ambos pueden compartir la misma confirmación. Si usa `RequiresNew`, puede confirmar su trabajo aunque el exterior termine abortando. Si usa `Suppress`, el bloque no participa en el ambiente. Una arquitectura que anida ámbitos sin establecer esta semántica puede producir resultados sorprendentes incluso si el código compila y los tests más sencillos pasan.

Dos DbContext no significan automáticamente dos transacciones distribuidas

Los objetos `DbContext` son unidades de trabajo, no equivalen necesariamente a **dos conexiones físicas durables**. El laboratorio de AceriaData utiliza dos contextos sobre una misma conexión SQL abierta para estudiar su coordinación sin exigir una infraestructura de transacciones distribuidas. Eso permite separar el concepto «varios contextos» de «varios recursos coordinados».

Incorporar conexiones físicas adicionales puede provocar una **promoción** que requiera coordinación distribuida. El comportamiento depende del proveedor y del entorno. En .NET moderno, el soporte de transacciones distribuidas de `System.Transactions` está sujeto a restricciones de plataforma; en particular, no debe diseñarse un laboratorio de uso general como si MSDTC estuviera disponible por defecto. La propiedad `DistributedIdentifier` de `Transaction.Current.TransactionInformation` proporciona una señal que puede investigarse, pero la interpretación debe acompañarse del conocimiento de las conexiones reales.

La pregunta útil no es «¿cuántos repositorios tengo?», sino «¿qué recursos participan, con qué conexión y bajo qué mecanismo transaccional?».

Niveles de aislamiento: lo que prometen y lo que no

`ReadCommitted` evita leer datos que otra transacción todavía no ha confirmado. No equivale a `ReadUncommitted` ni asegura por sí solo que una lectura repetida devuelva siempre el mismo resultado. En SQL Server, los detalles cambian según la configuración de la base, incluida la posibilidad de leer mediante versionado de filas en el modo correspondiente.

`Snapshot` permite observar una imagen consistente de los datos usando versionado cuando la base está configurada para soportarlo. Puede reducir bloqueos de lectura en determinados escenarios, pero no significa «sin bloqueos» ni evita automáticamente todos los conflictos de escritura. También introduce coste de version store. No deberíamos seleccionar un aislamiento por su nombre: debemos relacionarlo con las anomalías que la operación puede tolerar.

Una transacción de SQL Server no controla el mundo exterior

Pensemos en este orden de operaciones:

1. Insertar una orden en SQL Server.
2. Publicar un mensaje en un sistema externo.
3. Detectar un error y abortar la transacción SQL.

El resultado posible es que el mensaje externo permanezca mientras la orden no queda confirmada. `TransactionScope` **no** convierte de forma automática una llamada HTTP, un archivo local o una cola ordinaria en un recurso transaccional coordinado. Esta limitación no es un defecto de EF Core: es una frontera entre sistemas.

Para coordinar base de datos y mensajes, el **patrón outbox** escribe el evento pendiente en la misma base y transacción que la información de negocio. Después, un proceso independiente lee los eventos pendientes y los publica. Como puede haber reintentos, el consumidor debe admitir duplicados o usar identificadores

idempotentes. Otras soluciones incluyen compensaciones y procesos tipo saga. No hay una receta universal: se elige según garantías, costes y tolerancia a estados intermedios.

Conclusión del punto. TransactionScope aporta coordinación ambiental, pero la facilidad sintáctica no elimina la necesidad de conocer conexiones, aislamiento, promoción y recursos externos. La consistencia entre sistemas es un problema arquitectónico que supera una transacción SQL local.

Experimento completo: dos contextos y una conexión física

El repositorio real `TransaccionesAmbientalesM5Repositorio.cs` del punto 5.5 ofrece `MostrarDosContextosAsync`, `MostrarRollbackSinCompleteAsync`, `MostrarOpcionesDeScope`, `MostrarReadCommitted`, `MostrarSnapshot`, `MostrarRecursoExternoNoTransaccional` y `MostrarSuppressFueraDeRollback`. La diferencia entre estos escenarios es deliberada: no debe inferirse que dos objetos `DbContext` impliquen **dos conexiones físicas independientes** o promoción distribuida obligatoria.

```
var opciones = new TransactionOptions
{
    IsolationLevel = IsolationLevel.ReadCommitted,
    Timeout = TimeSpan.FromSeconds(30)
};
using var scope = new TransactionScope(
    TransactionScopeOption.Required,
    opciones,
    TransactionScopeAsyncFlowOption.Enabled);
await using var conexion = new SqlConnection(connectionString);
await conexion.OpenAsync();
// En el proyecto, dos contextos participan sobre esta misma conexión.
Console.WriteLine(Transaction.Current is not null);
await Task.Yield();
Console.WriteLine(Transaction.Current is not null);
// scope.Complete() sólo después de confirmar todas las operaciones.
```

El bloque enseña la creación y propagación del ámbito, pero **no ejecuta por sí solo las escrituras de los dos contextos**. Para esa parte se estudia el método real del repositorio, que recibe la conexión compartida y verifica la persistencia desde fuera. `Complete()` marca el éxito del scope; disponer de un scope sin llamarlo aborta la transacción participativa.

Variante	Pregunta experimental	Evidencia adecuada
Required	¿Se utiliza el ambiente exterior?	<code>Transaction.Current</code> y resultado al salir del scope.
RequiresNew	¿Se crea un límite de confirmación independiente?	Dos ámbitos y comportamiento ante el rollback exterior.
Suppress	¿Se ejecuta fuera del ambiente?	<code>Transaction.Current == null</code> dentro del bloque.
ReadCommitted	¿Se evitan lecturas sucias?	Dos transacciones coordinadas y visibilidad real.
Snapshot	¿Las lecturas usan una versión coherente?	Opción de aislamiento y configuración <code>ALLOW_SNAPSHOT_ISOLATION</code> .
Recurso externo	¿Lo deshace un rollback SQL?	SQL revertido; simulación del efecto externo conservada.

Promoción y plataforma. Observe la propiedad `DistributedIdentifier` de `Transaction.Current.TransactionInformation` conexiones y recursos realmente enrolados. Una única conexión compartida no prueba que la arquitectura sea apta para transacciones distribuidas; éstas pueden exigir Windows y MSDTC. Snapshot tampoco equivale a ausencia universal de bloqueos ni garantiza ausencia de conflictos.

Punto 5.6 – Migraciones en producción: estrategias y despliegue

Del comando de desarrollo al despliegue controlado

Durante el desarrollo, `dotnet ef database update` puede ser suficiente para poner una base local al día. En producción, una migración deja de ser una operación privada: modifica un recurso compartido y puede afectar a

varias versiones de la aplicación, procesos automáticos y datos reales. El objetivo de un despliegue profesional no es simplemente «que se ejecute el comando», sino lograr que el **esquema esperado** aparezca en el destino de forma observable y recuperable.

En AceriaData debemos conservar una cadena conocida de migraciones, desde los módulos anteriores hasta M5_5_2_ConcurrencyTokens. El proyecto Infrastructure contiene el DbContext y las migraciones; el proyecto Console actúa como startup para las herramientas. Esa separación determina cómo se invoca la CLI.

Cuatro estrategias y sus contextos

Estrategia	Ventaja principal	Cuidado necesario
Script SQL revisado	Un DBA o proceso de aprobación ve las instrucciones antes de ejecutarlas.	Hay que controlar versión, credenciales y aplicación.
Migration bundle	Entrega un ejecutable preparado para aplicar migraciones.	Sigue necesitando conexión, permisos y coordinación.
dotnet ef database update en job	Sencillo cuando el agente tiene SDK y acceso controlado.	No conviene depender de ejecuciones manuales improvisadas.
Database.Migrate() en la aplicación	Puede resultar cómodo en entornos pequeños.	En múltiples réplicas añade riesgos de carreras y eleva permisos del runtime.

No basta con ordenar estas opciones de mejor a peor. Un equipo con un DBA puede preferir SQL revisable; un pipeline controlado puede usar un bundle; un laboratorio puede usar IMigrator para estudiar el servicio de EF Core. La elección responde a restricciones operativas, no a una regla universal.

Construcción del artefacto antes del despliegue

Estos comandos son **ejemplos de generación** para ejecutarse desde el directorio del punto del proyecto, con rutas relativas a AceriaData y las herramientas EF instaladas:

SQL revisable desde la cadena de migraciones.

```
dotnet ef migrations script --idempotent `
  --project src/AceriaData.Infrastructure `
  --startup-project src/AceriaData.Console `
  --configuration Release `
  --output deployment/artifacts/aceria-idempotent.sql
```

Ejecutable de migraciones; no incluye secretos de producción.

```
dotnet ef migrations bundle `
  --project src/AceriaData.Infrastructure `
  --startup-project src/AceriaData.Console `
  --configuration Release `
  --output deployment/artifacts/aceria-efbundle.exe
```

Un script generado debe revisarse como cualquier otro cambio que pueda afectar datos: instrucciones de borrado, modificaciones de nulabilidad, valores por defecto, creación de índices y operaciones sobre tablas voluminosas. Si se detecta una migración con una transformación de datos costosa, puede ser preferible separar el cambio de esquema de un **backfill** controlado.

MigrationsAssembly: dónde busca EF Core las migraciones

Cuando DbContext está en Infrastructure y el programa de inicio en Console, la configuración puede indicar explícitamente el ensamblado de migraciones. Esto evita que un comando ejecute correctamente el proyecto de inicio pero no encuentre la cadena esperada. El manual y la práctica deben utilizar las rutas que existen en cada punto, no rutas de un proyecto anterior que se supone equivalente.

El historial no es una tabla desechable

EF Core registra las migraciones aplicadas en __EFMigrationsHistory. Esa tabla responde a una pregunta concreta: «¿qué identificadores de migración considera aplicados esta base?». No ofrece por sí sola una prueba completa de que nadie haya modificado manualmente el esquema, pero es un componente crítico para calcular qué falta.

Cambiar el nombre mediante `MigrationsHistoryTable(...)` en una base ya desplegada, sin trasladar el historial, puede hacer que EF Core pierda la referencia a los cambios aplicados. Un script posterior podría intentar volver a crear objetos existentes. Por ese motivo *AceriaData* conserva el nombre histórico. Personalizarlo es una decisión válida cuando se planifica desde el inicio o se acompaña de una transición explícita y comprobada.

El preflight: comprobar antes de cambiar

Un pipeline de despliegue serio realiza verificaciones antes del primer cambio irreversible: conexión al destino correcto, versión del artefacto, historial actual, permisos de la identidad que ejecuta, espacio, configuración de backups y compatibilidad temporal entre versiones de la aplicación. Si una migración es potencialmente bloqueante, también hay que conocer el tamaño de la tabla y la ventana operativa disponible.

La necesidad de soportar dos versiones del código durante una publicación escalonada introduce el patrón **expand/contract**. En una primera versión se añaden columnas o estructuras compatibles; después se adapta el código nuevo para utilizarlas; y sólo cuando ya no existen consumidores antiguos se eliminan los elementos obsoletos. Este orden puede exigir varias entregas, pero evita que una migración rompa inmediatamente aplicaciones que todavía están ejecutándose.

Verificar que el esquema realmente cambió

Después de aplicar el artefacto, el despliegue debe comprobar **dos planos**: que el historial contiene las migraciones esperadas y que las columnas, índices o restricciones relevantes existen en el esquema físico. A eso se añaden pruebas de humo de las rutas funcionales afectadas. El mero código de salida 0 de una herramienta es útil, pero no demuestra por sí solo la compatibilidad de la aplicación con el esquema desplegado.

AceriaData tiene un ejemplo especialmente claro: la migración del token de concurrencia. La comprobación posterior debe verificar la columna que representa el token y después ejecutar una operación que la utilice. Si el modelo de EF Core espera `RowVersion` pero la base no tiene esa columna, la configuración del código no resuelve la discrepancia.

Recuperación: por qué Down no equivale a backup

Generar un script inverso puede ser útil para comprender la implementación de los métodos Down, pero **deshacer el esquema no implica recuperar los datos**. Si una migración eliminó una columna, revertir su definición no devuelve necesariamente los valores anteriores. Un procedimiento de recuperación puede implicar restauración verificada, corrección hacia delante o reprocesamiento de datos. Debe prepararse antes del despliegue y ajustarse al riesgo real.

Conclusión del punto. En producción se despliegan **artefactos identificables** mediante una operación coordinada: revisión, preflight, ejecución, verificación y recuperación. El código de migración no sustituye esa disciplina.

Aplicar migraciones mediante el servicio real IMigrator

En *AceriaData* el código real de 5.6 obtiene `IMigrator` con `context.GetService<IMigrator>()` y llama a `MigrateAsync()` para un escenario **controlado de laboratorio**. Después consulta migraciones aplicadas y pendientes y verifica que la tabla `__EFMigrationsHistory` existe. Es una demostración de la API, no una recomendación para que varias réplicas de producción alteren el esquema simultáneamente al arrancar.

// Dentro de un caso de uso con AceriaDbContext ya configurado.

```
var migrador = context.GetService<IMigrator>();
await migrador.MigrateAsync();
var aplicadas = (await context.Database.GetAppliedMigrationsAsync()).ToArray();
var pendientes = (await context.Database.GetPendingMigrationsAsync()).ToArray();
Console.WriteLine($"Aplicadas: {aplicadas.Length}");
Console.WriteLine($"Pendientes: {pendientes.Length}");
Console.WriteLine($"Última: {aplicadas.LastOrDefault()}");
```

Este fragmento requiere `using Microsoft.EntityFrameworkCore.Infrastructure;` y `using Microsoft.EntityFrameworkCore;` además del contexto y los servicios necesarios. Es especialmente importante que la cadena de conexión apunte a una **base de ensayo**. En producción, una responsabilidad de despliegue separada y con permisos específicos debería aplicar un script revisado o un bundle aprobado.

Proceso mínimo de entrega: generar artefacto desde la migración real, registrar su versión, realizar comprobaciones previas de conectividad/permisos/espacio y backups según el riesgo, aplicar en ventana controlada, comprobar `__EFMigrationsHistory`, verificar columnas y ejecutar un smoke test. En caso de fallo, la recuperación exige evaluar qué cambios llegaron a ejecutarse y si un Down perdería información; un downgrade no es sinónimo de restauración.

El historial es parte de la compatibilidad. Si una base lleva años usando `__EFMigrationsHistory`, cambiar

MigrationsHistoryTable sin trasladar ese historial puede hacer que EF Core crea pendientes migraciones físicamente aplicadas. El laboratorio evita este error conservando el nombre heredado.

Punto 5.7 – Migraciones idempotentes y scripts SQL

Una flota no siempre comparte la misma versión de esquema

Supongamos tres instalaciones de AceriaData: una ya tiene la migración de concurrencia, otra está dos versiones por detrás y una tercera acaba de crearse. Una secuencia SQL que asume un estado inicial fijo no se puede aplicar indiscriminadamente a todas. Un script **idempotente** generado por EF Core incluye comprobaciones basadas en el historial para omitir las migraciones que ya figuran como aplicadas dentro de la cadena conocida.

Eso no significa que el script sea capaz de reparar cualquier base inconsistente. Su promesa es más limitada: manejar determinados **estados válidos de partida** según los identificadores de migración. Si alguien eliminó manualmente una columna pero dejó intacta __EFMigrationsHistory, el script puede considerar que esa migración ya está aplicada y no recrear la columna perdida.

Tres clases de scripts que no deben confundirse

1. Script de la cadena completa: requiere conocer el estado de partida.

```
dotnet ef migrations script --project $Infrastructure --startup-project $Startup `
  --output $Completo
```

2. Script con guardas por historial para varios estados válidos.

```
dotnet ef migrations script --idempotent `
  --project $Infrastructure --startup-project $Startup `
  --output $Idempotente
```

3. Script entre dos migraciones que existen realmente.

```
dotnet ef migrations script M2_2_12_Architecture M5_5_2_ConcurrencyTokens `
  --project $Infrastructure --startup-project $Startup `
  --output $DeRango
```

Aquí \$Infrastructure, \$Startup y las rutas de salida son variables PowerShell **que deben definirse antes**. Los nombres de migración pertenecen a la cadena documentada de AceriaData; conviene comprobar su presencia exacta en el proyecto antes de ejecutar un comando sobre una base concreta.

Para estudiar operaciones Down, la CLI permite especificar el rango en sentido inverso. Esa posibilidad es útil como análisis de SQL, pero un script generado para ir hacia atrás puede ejecutar borrados y transformaciones destructivas.

Qué es una prueba de idempotencia convincente

La secuencia tiene que ser observable y reproducible:

1. Crear una base **aislada** y conocida, nunca la base habitual del alumno.
2. Obtener un único archivo SQL idempotente y conservarlo sin modificar.
3. Aplicar el archivo sobre la base y exigir que el ejecutable termine correctamente.
4. Comprobar la última migración y los objetos físicos que ésta debe crear.
5. Ejecutar **el mismo archivo** una segunda vez.
6. Comparar el historial y el esquema: no deben aparecer migraciones duplicadas ni cambios inesperados.

Un ejemplo de invocación, cuando el servidor, base y archivo ya están correctamente configurados, es:

```
$ErrorActionPreference = 'Stop'
```

sqlcmd es un ejecutable externo: comprobar su código de salida.

```
& sqlcmd -S $Server -d $Database -E -I -b -i $Idempotente
if ($LASTEXITCODE -ne 0) { throw 'Falló la primera aplicación.' }
```

```
$count1 = [int](Read-Scalar 'SELECT COUNT(*) FROM dbo.__EFMigrationsHistory;')
```

```
& sqlcmd -S $Server -d $Database -E -I -b -i $Idempotente
if ($LASTEXITCODE -ne 0) { throw 'Falló la segunda aplicación.' }
```

```
$count2 = [int](Read-Scalar 'SELECT COUNT(*) FROM dbo.__EFMigrationsHistory;')
if ($count1 -ne $count2) { throw 'La historia cambió en la segunda pasada.' }
```

Read-Scalar es una **función auxiliar del laboratorio**, no un comando integrado de PowerShell. El fragmento explica la comparación y no debe presentarse como un script independiente que se ejecuta sin preparar la base, las variables y la función. La opción `-b` ayuda a propagar errores de SQL; `-I` establece `QUOTED_IDENTIFIER` conforme a las necesidades del escenario utilizado. El control de `$LASTEXITCODE` es importante porque `$ErrorActionPreference = 'Stop'` no cubre automáticamente todos los fallos de ejecutables externos.

Comprobar historia y esquema son pruebas distintas

Podemos encontrar tres estados relevantes:

Historial	Esquema físico	Interpretación
Esperado	Esperado	Evidencia consistente para los objetos revisados.
Esperado	Falta una columna esperada	Existe una discrepancia o <i>drift</i> ; no basta con repetir el script.
Incompleto	Hay objetos de una migración que no figura aplicada	Investigar despliegue incompleto o cambio manual antes de avanzar.

Esta tabla ayuda a evitar una reacción peligrosa: borrar registros de `__EFMigrationsHistory` para «forzar» que EF repita operaciones. El historial debe tratarse como parte de la cadena de despliegue, no como una caché que se vacía sin consecuencias.

Artefacto inmutable y versiones conocidas

Cuando un script ha sido revisado y aprobado, conviene asociarlo a una versión de entrega y mantenerlo inmutable. Si se vuelve a generar con una cadena de migraciones distinta, ya no es el mismo artefacto aunque tenga el mismo nombre de archivo. Esta disciplina facilita investigar qué SQL se aplicó a cada entorno y comparar incidentes con el artefacto exacto.

Límites del concepto

Idempotencia no implica reversibilidad; un script que evita aplicar dos veces una migración no recupera información eliminada por esa migración. Tampoco implica compatibilidad automática entre código viejo y nuevo ni garantiza seguridad bajo ejecuciones simultáneas descoordinadas. Es una **propiedad de aplicación de una cadena histórica**, útil, pero parte de un proceso de despliegue más amplio.

Conclusión del punto. Una prueba real de idempotencia ejecuta dos veces el **mismo** artefacto y comprueba resultados. Leer guardas IF sin ejecutar ni verificar el esquema es evidencia insuficiente.

Prueba más completa del mismo artefacto: historial y columna

El escenario de 5.7 se apoya en nombres reales de la cadena: `M2_2_12_Architecture` y `M5_5_2_ConcurrencyTokens`. El siguiente bloque muestra cómo consultar **tanto la historia como una columna física**, sin recurrir a una función `Read-Scalar` indefinida. Supone que las variables `$Server`, `$Database` y `$Idempotente` ya se inicializaron y que la base es desechable.

```
# El mismo SQL tiene que aplicarse dos veces sin modificaciones.
$sql = 'SET NOCOUNT ON; SELECT COUNT(*) FROM dbo.__EFMigrationsHistory;'
$col = "SELECT COUNT(*) FROM sys.columns c " +
      "JOIN sys.tables t ON c.object_id=t.object_id " +
      "WHERE t.name='OrdenesFabricacion' AND c.name='RowVersion';"

function Escalar([string]$consulta) {
    $resultado = & sqlcmd -S $Server -d $Database -E -h -l -W -Q $consulta -b
    if ($LASTEXITCODE -ne 0) { throw "Fallo sqlcmd: $consulta" }
    return [int]($resultado | Where-Object { $_.Trim() } | Select-Object -Last 1)
}

& sqlcmd -S $Server -d $Database -E -I -b -i $Idempotente
if ($LASTEXITCODE -ne 0) { throw 'Primera aplicación fallida' }
$historial = Escalar $sql; $columna1 = Escalar $col
& sqlcmd -S $Server -d $Database -E -I -b -i $Idempotente
if ($LASTEXITCODE -ne 0) { throw 'Segunda aplicación fallida' }
$historia2 = Escalar $sql; $columna2 = Escalar $col
if ($historial -ne $historia2 -or $columna1 -ne $columna2 -or $columna2 -ne 1) {
    throw 'El historial o el esquema no permanecieron consistentes'
}
```

La segunda ejecución **no garantiza por sí sola** que el esquema esté correcto: por eso también consultamos `sys.columns`. El conteo debe complementarse con la identificación de la **última migración esperada**, ya que dos historias distintas pueden tener el mismo número de filas. Además, esta rutina no repara drift y no justifica usar una base de producción como campo de pruebas.

Por qué los pipelines importan. La práctica incluye GitHub Actions, Azure DevOps y despliegue sobre varias bases; esos pipelines no deben almacenar contraseñas ni cadenas de conexión en el repositorio. Deben recibir credenciales como secretos, ejecutar un artefacto aprobado, registrar su identificador y detenerse si falla cualquiera de las comprobaciones. Ante varios destinos, cada base debe tener su propio historial y comprobaciones, sin concluir que el éxito de la primera garantiza las restantes.

Punto 5.8 — Migraciones en equipos: conflictos y buenas prácticas

El problema no es el nombre de un archivo

Dos desarrolladores, Ana y Bruno, parten del mismo snapshot de AceriaData. Ana añade una propiedad al modelo y genera la migración A. Sin haber incorporado ese cambio, Bruno añade otra propiedad y genera la migración B. Cada migración es coherente respecto del **snapshot que existía cuando fue generada**. Al integrar las dos ramas, la secuencia puede contener artefactos que describen historias diferentes.

El conflicto no se reduce a ordenar timestamps. Las migraciones incluyen código Up y Down, un archivo `.Designer.cs` con metadatos del modelo objetivo y un snapshot que representa el estado reconocido por EF Core. Renombrar la migración de Bruno para que parezca posterior a la de Ana **no regenera** los metadatos que desconocían la propiedad de Ana.

Reconstruir el punto común

Antes de resolver nada debemos identificar el ancestro de las dos ramas: última migración y snapshot comunes. A continuación se distingue el código que pertenece al cambio de Ana del correspondiente a Bruno. Sin esa separación podemos borrar trabajo válido o introducir una migración que compile pero describa incorrectamente la evolución del modelo.

El laboratorio utiliza propiedades de demostración `EquipoRevisionA` y `EquipoRevisionB` para hacer visible la divergencia. La rama B paralela no debería incluir `EquipoRevisionA` en los metadatos generados, puesto que su autor todavía no la conocía. Comprobar el `.Designer.cs` permite demostrar el problema más allá de que Git muestre o no un conflicto textual.

Si la migración B todavía es privada

Cuando Bruno **no ha compartido ni aplicado** su migración en entornos comunes, puede retirarla de forma controlada mientras conserva el cambio que hizo al modelo. Después incorpora la rama de Ana, de modo que el snapshot actualizado ya contiene A, y regenera su propia migración B. La nueva secuencia expresa A y después B.

La operación puede implicar `dotnet ef migrations remove` cuando es seguro hacerlo, integración de cambios y una nueva ejecución de `dotnet ef migrations add` sobre el modelo combinado. No se trata de borrar archivos manualmente hasta lograr que el build pase: hay que mantener sincronizados código, migraciones y snapshot.

Una representación del proceso sería:

Momento	Snapshot conocido al generar B	Resultado
B paralela	Estado inicial, sin A	B desconoce la modificación de Ana.
B regenerada tras integrar A	Estado inicial + A	B se calcula sobre el modelo ya actualizado.

Cómo verificar la cadena fusionada

Una vez regenerada, comprobamos tres aspectos diferentes. **Primero**, el modelo actual debe coincidir con el snapshot: en EF Core 8 puede utilizarse `dotnet ef migrations has-pending-model-changes`. **Segundo**, la cadena debe poder aplicarse sobre una base SQL Server aislada mediante las migraciones reales, no sólo compilar. **Tercero**, se comprueba físicamente que existen las columnas de Ana y Bruno y que el historial registra las migraciones correspondientes.

Desde el directorio de la solución AceriaData correspondiente.

```
dotnet ef migrations has-pending-model-changes `
  --project src/AceriaData.Infrastructure `
  --startup-project src/AceriaData.Console
```

Ejecutar solamente contra una base aislada preparada para el ejercicio.

```
dotnet ef database update `
  --project src/AceriaData.Infrastructure `
  --startup-project src/AceriaData.Console
```

La segunda operación requiere verificar a **qué base apunta la cadena de conexión** antes de ejecutarla. Por esta razón el curso usa bases desechables específicas para los escenarios de migración.

Si la migración ya se publicó

La decisión cambia cuando una migración fue compartida, ejecutada por compañeros o aplicada en un entorno. Reescribirla unilateralmente puede provocar que dos bases registren el mismo identificador pero hayan recibido SQL diferente. En ese caso, la solución suele pasar por una **migración correctiva hacia delante** o por una operación coordinada de reversión cuando sea segura. No existe una prohibición absoluta de eliminar toda migración, sino una frontera de responsabilidad: una migración local es un artefacto privado; una migración publicada forma parte de una historia compartida.

Qué revisar en un pull request

Las revisiones no deberían limitarse a detectar conflictos en `ModelSnapshot.cs`. Conviene revisar Up, Down, el diseñador, las propiedades y relaciones afectadas, los nombres de columnas, la nulabilidad, los valores por defecto, las operaciones potencialmente caras y la posible pérdida de datos. Una fusión textual limpia puede seguir teniendo un resultado semántico incorrecto.

Conclusión del punto. Integrar migraciones en equipo exige reconstruir la historia de cambios del modelo. El criterio fundamental es conservar la coherencia entre **Git**, **snapshot**, **identificadores de migración** y **esquema físico**.

Dos laboratorios distintos: migraciones paralelas y ramas Git

La teoría utiliza `EquipoRevisionA` y `EquipoRevisionB` para aislar el problema técnico de metadatos en dos copias temporales. La práctica extensa presenta otro recorrido, con ramas `feature/AddDetalleOrden` y `bugfix/AddIndiceNumeroOrden`. **No deben mezclarse como si fueran los nombres de una única ejecución:** ambos ilustran integración de cambios, pero requieren pasos de modelo, migraciones y verificación diferentes.

Secuencia	Laboratorio controlado	Práctica de Git
Estado común	Dos copias del mismo snapshot	Ramas desde un ancestro compartido
A crea cambio	Propiedad <code>EquipoRevisionA</code>	Introducción de <code>DetalleOrden</code>
B crea cambio	Propiedad <code>EquipoRevisionB</code>	Índice sobre <code>NumeroOrden</code>
Riesgo	Designer B no conoce el modelo A	Snapshot/metadatos divergentes al integrar
Reparación	Regenerar migración B privada sobre A	Reconciliar snapshot y regenerar migración privada cuando corresponda
Evidencia	Designer con A+B, ambas columnas, historial	Diff revisado, índice/tabla finales y cadena aplicada

Antes de eliminar o regenerar B hay que demostrar que **no ha llegado a ninguna base compartida**. `dotnet ef migrations remove` tiene una semántica distinta de borrar manualmente un `Designer.cs`: actualiza los artefactos de migración y snapshot según el estado conocido. Si B ya está publicada, es posible que el equipo necesite una migración **correctiva hacia delante** en lugar de reescribir la historia.

Criterio de cierre de 5.8. Son necesarias cuatro verificaciones independientes: migraciones ordenadas en la historia, metadata del designer regenerada sobre A, `has-pending-model-changes` sin diferencias y esquema final comprobado físicamente en LocalDB aislada. La ausencia de conflictos Git no basta para afirmar que el modelo EF quedó bien integrado.

Punto 5.9 – Repository y Unit of Work en aplicaciones empresariales

El problema arquitectónico que queremos resolver

AceriaData utiliza una arquitectura en capas: **Domain** contiene entidades y reglas de dominio; **Application** expresa casos de uso; **Infrastructure** implementa persistencia; y **Console** construye el proceso y las dependencias. Cuando un caso de uso necesita consultar órdenes, podríamos inyectarle directamente `AceriaDbContext`. La pregunta no es si EF Core puede hacerlo —puede—, sino qué dependencia queremos admitir en esa capa.

`DbContext` ya representa una unidad de trabajo en la práctica: sigue cambios y los confirma mediante `SaveChanges`. `DbSet<TEntity>` ya ofrece operaciones parecidas a las de un repositorio. Por tanto, añadir `Repository` y `Unit of Work` propios **no es un requisito técnico de EF Core**. Sólo tiene sentido si expresan mejor las responsabilidades o proporcionan una frontera útil para el diseño.

En `AceriaData` se elige una frontera: los casos de uso de `Application` conocen **interfaces propias** y `Infrastructure` conoce EF Core. De esta manera las políticas de negocio pueden probarse con colaboradores falsos o mocks, mientras el comportamiento del proveedor se verifica en tests de integración independientes.

Un repositorio que expresa intención

Comparemos dos interfaces conceptuales:

// Abstracción genérica: válida si sus operaciones comunes aportan valor.

```
public interface IRepository<T> where T : class
{
    T? ObtenerPorId(int id);
    List<T> ObtenerTodas();
    void Agregar(T entidad);
    void Eliminar(T entidad);
}
```

// El contrato real de AceriaData incluye estas operaciones, entre otras:

```
public interface IOrdenRepositorio : IRepository<OrdenFabricacion>
{
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    List<OrdenFabricacion> ObtenerPendientesPorCliente(string cliente);
    // ... Otras consultas específicas del proyecto.
}
```

La primera interfaz no está mal por ser genérica: puede evitar duplicación de operaciones simples. Pero si termina replicando todos los métodos de `DbSet`, `Include`, `AsNoTracking`, `ExecuteUpdate` y opciones del proveedor, puede añadir una capa que renombra EF Core sin desacoplar de verdad. La segunda expresa operaciones que tienen significado para el caso de uso, aunque también debe mantenerse proporcionada para no crear un método distinto por cada variación trivial de una consulta.

El siguiente caso de uso simplificado **sí corresponde a una clase existente** en `M05/PROYECTO/5.9`: `Application` recibe el contrato de repositorio por inyección, filtra las órdenes pendientes sin conocer `Infrastructure` y delega el registro. La implementación concreta que traduce las consultas a EF Core queda detrás del contrato:

```
public sealed class OrdenesConsultaM5Service
{
    private readonly IOrdenRepositorio _repositorio;

    public OrdenesConsultaM5Service(IOrdenRepositorio repositorio) =>
        _repositorio = repositorio;

    public List<OrdenFabricacion> ObtenerPendientes() =>
        _repositorio.ObtenerTodas()
            .Where(o => o.Estado == "Pendiente")
            .ToList();

    public void Registrar(OrdenFabricacion orden) =>
        _repositorio.Agregar(orden);
}
```

Las cadenas y reglas de filtrado son deliberadamente simples para explicar el patrón. En producción, las constantes de estado y validaciones deben corresponder con el dominio efectivo de `AceriaData`. Además, si el resultado va a mostrarse y no modificarse, una proyección a DTO y `AsNoTracking` pueden reducir materialización; la forma

elegida depende de quién consumirá la respuesta.

Por qué exponer IQueryable puede atravesar la frontera

Una interfaz como `IQueryable<OrdenFabricacion>` `ObtenerOrdenes()` permite que `Application` añada filtros y proyecciones. Eso ofrece flexibilidad, pero obliga a quien consume el contrato a comprender características de LINQ traducido a SQL, carga de navegaciones, tracking y proveedor. La capa exterior ha quedado parcialmente acoplada a la semántica de EF Core aunque no cite directamente sus paquetes.

Este compromiso puede aceptarse dentro de una capa que ya trabaja con EF Core. Pero si la finalidad explícita es **ocultar Infraestructura a Application**, es importante reconocer esa fuga conceptual y preferir operaciones con intención, DTOs o especificaciones diseñadas para no exponer detalles innecesarios.

Unit of Work: coordinar, no duplicar

Imaginemos un caso de uso que registra una orden y su único detalle asociado. Si cada repositorio utiliza un `DbContext` diferente y ejecuta `SaveChanges` internamente, el caso de uso pierde control sobre cuándo se confirma la unidad completa. Una `IUnidadDeTrabajo` puede proporcionar acceso coordinado a varios repositorios sobre el **mismo contexto** y una operación de confirmación `Guardar()`.

```
public interface IUnidadDeTrabajo : IDisposable
{
    IOrdenRepositorio Ordenes { get; }
    IDetalleOrdenRepositorio Detalles { get; }
    int Guardar();
}
```

La interfaz anterior reproduce la estructura de `IUnidadDeTrabajo` de `AceriaData`: `Ordenes` y `Detalles` se coordinan y `Guardar()` define el momento de confirmación. El caso de uso real registra una orden y un detalle, llama a `_unidad.Guardar()` y los recupera para comprobar que existen. Debemos explicar que estamos exponiendo una capacidad de `DbContext.SaveChanges()` mediante una abstracción arquitectónica, no inventando una transacción adicional.

La duración de DbContext forma parte del diseño

Un `DbContext` no debe compartirse concurrentemente entre hilos como si fuera un singleton. Su `ChangeTracker` almacena estado y sus operaciones no están diseñadas para acceso concurrente desde varias tareas. En aplicaciones basadas en peticiones, un scope suele corresponder con una unidad de trabajo. En workers, herramientas de consola y tareas de duración variable puede ser más apropiado crear contextos bajo demanda mediante `IDbContextFactory<AceriaDbContext>`.

```
// Fragmento de un servicio que recibe una factory por inyección.
await using var db = await factory.CreateDbContextAsync(ct);
var pendientes = await db.OrdenesFabricacion
    .AsNoTracking()
    .Where(o => o.Estado == "Pendiente")
    .ToListAsync(ct);
```

La factory crea nuevas unidades; **no convierte al contexto en seguro para acceso concurrente**, ni significa que debamos crear uno por cada método sin considerar la operación de negocio. Cada contexto debe disponerse al terminar su unidad de trabajo.

Qué puede demostrar un test con Moq

Si `Application` depende de `IOrdenRepositorio`, un test unitario puede verificar que llama al método esperado o que decide correctamente según la respuesta simulada. Un mock del repositorio **no prueba** que EF Core genere SQL correcto, que las relaciones estén bien configuradas o que SQL Server aplique una clave foránea. Intentar simular `DbSet` y toda la traducción de LINQ suele crear una imitación menos fiel que el proveedor real.

Conclusión del punto. Repository y Unit of Work son herramientas arquitectónicas **opcionales**. En `AceriaData` tienen sentido cuando mantienen dependencias claras, expresan operaciones del dominio y permiten separar tests unitarios de tests de infraestructura. Su coste es el mantenimiento de contratos y adaptadores adicionales.

Un caso real de AceriaData, sin cambiar la cardinalidad

El modelo **no admite dos `DetalleOrden` distintos para una misma `OrdenFabricacion` a través de la navegación**: la propiedad `OrdenFabricacion.Detalle` es singular. El caso de uso real `RepositorioUnidadTrabajoM5UseCase` crea **una orden y un detalle**, y utiliza `_unidad.Ordenes.Agregar(orden)`, `_unidad.Detalles.Agregar(detalle)` y `_unidad.Guardar()`.

```
// Fragmento de la unidad de trabajo real; se omiten datos obligatorios
// del detalle, que el caso de uso original inicializa antes de agregar.
_unidad.Ordenes.Agregar(orden);
_unidad.Detalles.Agregar(detalle);
int filas = _unidad.Guardar();
var recuperada = _unidad.Ordenes.ObtenerPorNumero(orden.NumeroOrden);
var detalleGuardado = recuperada is null ? null :
    _unidad.Detalles.ObtenerPorOrden(recuperada.Id);
```

Este ejemplo presupone que orden y detalle existen y que detalle.Orden = orden. Una sola llamada a Guardar() coordina lo pendiente en el DbContext compartido. El método se llama **Guardar()**, no GuardarAsync(), en el contrato versionado; una futura API asíncrona sería una evolución arquitectónica explícita, no una capacidad que ya debamos atribuir al código.

Para validar el valor de la abstracción se separan **dos preguntas**: un test con Moq puede comprobar que OrdenesConsultaM5Service llama una vez a IOrdenRepositorio.ObtenerTodas() y filtra correctamente; una prueba de integración necesita el DbContext y SQL Server para demostrar persistencia, relaciones y transacciones efectivas. El proyecto contiene RepositoryPatternTests con esas aserciones de colaboración.

Punto 5.10 – Logging y diagnóstico de Entity Framework Core

Una incidencia: guardar una orden tarda más de lo esperado

Un responsable informa de que una orden «a veces tarda mucho en guardarse». Una captura de pantalla no indica si el retraso se produjo al abrir una conexión, generar SQL, esperar un bloqueo, ejecutar una consulta, confirmar una transacción o escribir en un servicio externo. Antes de modificar código debemos reconstruir la operación con **señales de observabilidad**.

Las señales se complementan, pero no son intercambiables. Un **log** describe un acontecimiento contextualizado; un evento de **DiagnosticSource** expone detalles técnicos dentro del proceso; un **EventCounter** ofrece una medida agregada; y un backend de **telemetría** almacena y correlaciona los datos que realmente recibe. La calidad de la observabilidad depende de usar cada mecanismo para responder una pregunta adecuada.

Paso 1: empezar con ILogger estructurado

Un mensaje concatenado puede leerse, pero es más difícil de filtrar automáticamente. Las plantillas de ILogger conservan propiedades separadas:

```
using (logger.BeginScope(new Dictionary<string, object>
{
    ["Modulo"] = "M05",
    ["Punto"] = "5.10"
}))
{
    logger.LogInformation(
        "Guardando orden {NumeroOrden} para {Cliente}",
        numeroOrden,
        cliente);

    // Aquí se ejecutaría el caso de uso que realiza la escritura.
}
```

Con un proveedor que soporte logging estructurado, NumeroOrden puede almacenarse como atributo consultable, no sólo como texto. El scope añade contexto compartido durante el bloque. Es importante decidir qué datos registrar: nombres de personas, cadenas de conexión y valores sensibles no deberían terminar indiscriminadamente en logs de producción.

La aplicación define categorías y niveles. Para EF Core pueden resultar especialmente útiles Microsoft.EntityFrameworkCore.Update y Microsoft.EntityFrameworkCore.Query. Una investigación puntual puede justificar registros detallados, pero dejar todo el proveedor en Debug permanente puede generar ruido, costes de almacenamiento y exposición innecesaria.

Paso 2: integrar Serilog sin perder la estructura

Serilog recibe eventos estructurados y puede escribirlos en consola, fichero o sistemas centralizados. La configuración de un archivo no debería crecer ilimitadamente; podemos imponer rotación diaria, límite por tamaño y retención:

```
var log = new LoggerConfiguration()
    .MinimumLevel.Information()
    .Enrich.FromLogContext()
    .WriteTo.Console()
    .WriteTo.File(
        "logs/aceria-.log",
        rollingInterval: RollingInterval.Day,
        fileSizeLimitBytes: 4096,
        rollOnFileSizeLimit: true,
        retainedFileCountLimit: 3)
    .CreateLogger();
```

Los límites pequeños de este ejemplo son deliberados para un **laboratorio de rotación**; no constituyen valores recomendados para producción. El número y tamaño apropiados dependen del volumen. Para demostrar que la configuración funciona, el laboratorio produce suficientes eventos para superar el límite, comprueba que aparecen varios archivos y verifica que la retención no permite crecimiento indefinido.

La integración con ILoggerFactory debe mantener la configuración coherente: no queremos registrar dos veces el mismo evento mediante proveedores duplicados. Tras terminar el proceso hay que disponer correctamente los proveedores y sinks que poseen recursos.

Paso 3: diferenciar SQL previsto del realmente ejecutado

ToQueryString() es útil para estudiar el SQL de una consulta LINQ **antes de ejecutarla**. No demuestra que se haya enviado al motor. Para observar comandos realmente ejecutados utilizamos categorías de logging de EF Core o un DbCommandInterceptor.

```
// Consulta: la representación SQL todavía no implica ejecución.
var consulta = db.OrdenesFabricacion
    .AsNoTracking()
    .Where(o => o.Estado == "Pendiente");
```

```
Console.WriteLine(consulta.ToQueryString());
```

```
// Esta operación sí ejecuta la consulta y genera eventos de comandos.
var ordenes = await consulta.ToListAsync();
```

Esta diferencia es esencial en problemas de consultas N+1, donde necesitamos **contar comandos reales**, y en conflictos de concurrencia, donde interesa conocer el UPDATE emitido por SaveChanges.

Paso 4: DiagnosticListener como fuente de eventos internos

EF Core publica eventos de diagnóstico a través de DiagnosticSource. Para escucharlos, primero se descubren listeners mediante DiagnosticListener.AllListeners y después se selecciona el listener de EF Core. Un observador debe conservar y liberar las suscripciones:

```
// Núcleo ilustrativo del observador; la clase completa implementa
// IObservable<DiagnosticListener>, IObservable<KeyValuePair<string, object?>>
// y IDisposable, y gestiona la lista de suscripciones.
public void OnNext(DiagnosticListener listener)
{
    if (listener.Name == "Microsoft.EntityFrameworkCore")
        _subscriptions.Add(listener.Subscribe(this, IsEnabled));
}

private static bool IsEnabled(
    string eventName, object? arg1, object? arg2) =>
    eventName.Contains("Command", StringComparison.Ordinal) ||
    eventName.Contains("SaveChanges", StringComparison.Ordinal);
```

Aquí _subscriptions pertenece al observador real. Un fragmento como éste ilustra el filtro, pero **no** debe etiquetarse como implementación completa si no muestra la suscripción a AllListeners, el tratamiento de eventos y el Dispose. Esas piezas forman parte del escenario real de AceriaData, y su importancia es conceptual: sin ellas podemos perder eventos o mantener observadores activos más tiempo del previsto.

El payload de un evento puede contener detalles específicos que necesitan interpretación. No es recomendable tratar todos los eventos como cadenas arbitrarias ni asumir que cualquier evento de un proveedor conserva la misma forma en todas las versiones.

Paso 5: EventCounters de EF Core 8

EF Core 8 expone contadores a través de EventSource con el nombre Microsoft.EntityFrameworkCore. Un EventListener puede habilitar los contadores e interpretar payloads que incluyen valores como Mean o Increment, según el tipo de contador y el evento recibido. También puede utilizarse la herramienta dotnet-counters para observar un proceso desde fuera.

Los contadores responden a preguntas sobre **tendencias agregadas del proceso**, por ejemplo si cambia la actividad de ciertas operaciones durante una carga. No identifican por sí solos qué orden o usuario causó un pico. Para ello necesitamos correlación con logs y, cuando proceda, trazas. Es importante no atribuir a EF Core 8 APIs de métricas publicadas sólo en versiones posteriores.

Paso 6: Application Insights y Azure Monitor

Implementación real en AceriaData. Este curso utiliza ActivitySource, OpenTelemetry y el exportador Azure.Monitor.OpenTelemetry.Exporter; **no** configura el punto 5.10 con AddApplicationInsightsTelemetryWorker. La infraestructura crea un TracerProvider que escucha el origen AceriaData y añade AddAzureMonitorTraceExporter() cuando existe la variable de entorno APPLICATIONINSIGHTS_CONNECTION_STRING. Sin esa variable, la clase devuelve null y el laboratorio deja la exportación remota deshabilitada. De esto **no** puede deducirse que los eventos hayan llegado a Azure: hay que comprobar transporte, configuración y recepción en el backend. AddApplicationInsightsTelemetryWorkerService() es otra ruta de instrumentación, no la que se ejecuta en este proyecto.

La distinción entre **producir un evento** y **confirmar que un backend remoto lo recibió** evita falsos positivos. Ejecutar TrackEvent demuestra que se invocó una API; para afirmar recepción remota deben revisarse canal, conexión, exportación, red y consulta en el backend.

En arquitecturas actuales también es posible instrumentar mediante **OpenTelemetry** y exportar a Azure Monitor u otros destinos. Esa opción amplía portabilidad y correlación, pero tampoco elimina la obligación de comprobar que los datos llegaron al destino elegido.

Reconstruir la incidencia original

Ahora podemos establecer una secuencia de investigación: asociar un identificador de correlación a la operación; observar cuándo comenzó y terminó el caso de uso; consultar comandos de EF Core para identificar qué se ejecutó; analizar los eventos relevantes de SaveChanges; examinar tendencias agregadas para detectar contención o volumen; y, si existe backend de telemetría, correlacionar la petición con otros servicios.

Si el problema está en el tiempo de una consulta, analizaremos su SQL y plan de ejecución. Si está en una transacción larga, reconstruiremos la frontera transaccional. Si el log contiene datos sensibles, corregiremos la instrumentación antes de ampliar la captura. **Observabilidad útil no es maximizar mensajes; es hacer posible contestar una pregunta con evidencias suficientes.**

Conclusión del punto. ILogger, Serilog, DiagnosticSource, EventCounters y Application Insights cumplen funciones distintas. Un diagnóstico sólido combina las señales necesarias y comprueba tanto el contenido como el ciclo de vida, el volumen y la privacidad de la instrumentación.

Secuencia de implementación real: ActivitySource, eventos, contadores y exportación

La composición del punto 5.10 utiliza LoggingDiagnosticM5Runner, EfDiagnosticObserver, EfEventCounterListener y AzureMonitorOpenTelemetry. Están en src/AceriaData.Console/ del punto 5.10. Son cuatro piezas con responsabilidades distintas, y la teoría debe mostrarlas conjuntamente.

```
// Esquema fiel a AzureMonitorOpenTelemetry.CreateFromEnvironment().
var connectionString = Environment.GetEnvironmentVariable(
    "APPLICATIONINSIGHTS_CONNECTION_STRING");
if (!string.IsNullOrEmpty(connectionString))
{
    var provider = Sdk.CreateTracerProviderBuilder()
        .AddSource("AceriaData")
        .AddAzureMonitorTraceExporter(options =>
            options.ConnectionString = connectionString)
        .Build();
    // Disponer provider al finalizar la aplicación.
}
```

Se requieren los espacios de nombres y paquetes OpenTelemetry/Azure Monitor que ya aparecen en el proyecto. El runner abre un Activity desde el ActivitySource denominado AceriaData, añade tags de módulo y punto, emite logs mediante ILogger y realiza consultas/escrituras que generan señales EF Core. **Si no hay cadena de**

conexión, no se construye el exportador remoto, lo que permite ejecutar el laboratorio sin exponer secretos ni depender de un tenant Azure.

El observador de eventos se suscribe a `DiagnosticListener.AllListeners`, selecciona el listener `Microsoft.EntityFrameworkCore` y filtra eventos cuyo nombre incluye `Command` o `SaveChanges`. Conserva cada suscripción para liberarla con `Dispose`; no basta con implementar `OnNext()` si nunca se activa la suscripción general.

```
// Patrón real de EventCounterListener en el proyecto.
if (eventSource.Name == "Microsoft.EntityFrameworkCore")
    EnableEvents(eventSource, EventLevel.LogAlways, EventKeywords.All,
        new Dictionary<string, string?> { ["EventCounterIntervalSec"] = "1" });
// OnEventWritten recibe payloads EventCounters y extrae Name + Mean
// o Increment, cuando el contador del proveedor contiene esos campos.
```

`EventCounters` es una señal **agregada** y su intervalo de emisión introduce espera: no equivale a capturar un comando SQL concreto. El programa del punto comprueba que ha visto eventos y contadores, y fuerza emisión de ficheros Serilog para verificar rotación y retención. Esto no demuestra recepción en Azure Monitor; para esa afirmación deben comprobarse credenciales, exportación y telemetría en el servicio remoto.

Fuente	Qué responde	Prueba adecuada
ILogger/Serilog	¿Qué operación registró cada mensaje?	Plantilla, propiedades y scope; ficheros rotados.
DiagnosticListener	¿Qué eventos EF se publicaron durante los comandos/guardados?	Suscripción activada y contador de eventos.
EventCounters	¿Qué tendencia agregada reporta EF Core 8?	EventListener, nombres y valores observados.
ActivitySource + OpenTelemetry	¿Cómo se agrupan trazas y spans?	Origen conectado a TracerProvider.
Azure Monitor	¿Llegó la telemetría al backend?	Evidencia de exportación y recepción remota.

Seguridad. `EnableSensitiveDataLogging` se mantiene desactivado por defecto y los logs normales no deben incluir credenciales ni secretos. La instrumentación de alto volumen requiere filtros y una política explícita de conservación.

Punto 5.11 – Testing con EF Core

Elegir una prueba que pueda demostrar la afirmación

Una prueba automatizada vale por la **pregunta concreta que responde**. Si queremos comprobar que un caso de uso rechaza una orden sin número, probablemente basta con una prueba unitaria. Si queremos comprobar que SQL Server impide insertar un detalle con una clave foránea inexistente, necesitamos un proveedor relacional que aplique esa restricción. Si queremos demostrar que `rowversion` se actualiza automáticamente, necesitamos **SQL Server**, porque ese comportamiento pertenece al motor.

Por eso no tiene sentido reducir el testing de EF Core a una elección absoluta entre mocks e integración. Las pruebas forman capas complementarias, cada una con su coste y su nivel de fidelidad.

Primera capa: unit tests de Application con xUnit y Moq

Cuando un caso de uso depende de `IOrdenRepositorio`, podemos sustituir ese colaborador por un mock y comprobar una decisión del caso de uso sin arrancar SQL Server. Por ejemplo, un caso que debe obtener pendientes puede verificar que llama al método previsto y transforma correctamente el resultado.

```
[Fact]
public void ObtenerPendientes_UsaLaAbstraccionSinBaseDeDatos()
{
    var repo = new Mock<IOrdenRepositorio>();
    repo.Setup(r => r.ObtenerTodas()).Returns(new List<OrdenFabricacion>
    {
        new() { NumeroOrden = "OF-1", Estado = "Pendiente" },
        new() { NumeroOrden = "OF-2", Estado = "EnProceso" },
        new() { NumeroOrden = "OF-3", Estado = "Pendiente" }
    });

    var servicio = new OrdenesConsultaM5Service(repo.Object);
    var pendientes = servicio.ObtenerPendientes();

    Assert.Equal(2, pendientes.Count);
    repo.Verify(r => r.ObtenerTodas(), Times.Once);
}
```

Este ejemplo se corresponde con un test real de `RepositoryPatternTests.cs`: el mock devuelve tres órdenes, `OrdenesConsultaM5Service` filtra las pendientes y la aserción comprueba que quedan dos. `Verify` garantiza además que se usó la interfaz una vez. La prueba es autocontenida **dentro de la clase de tests con los using pertinentes** y no necesita SQL Server. Esto no significa que se hayan comprobado traducción SQL ni restricciones de base de datos.

Esta capa no debe intentar simular el traductor LINQ mediante mocks de `DbSet`. Ese traductor pertenece al proveedor; sustituirlo por expresiones en memoria puede hacer pasar una prueba que falla en SQL Server.

Segunda capa: diferencias de proveedor

EF Core ofrece `InMemory` para pruebas ligeras que no dependen de semántica relacional. Pero **InMemory no es un motor SQL**: no representa fielmente restricciones de claves foráneas, transacciones y traducción de consultas. `SQLite` sí es relacional y puede aplicar restricciones, pero sigue usando otro dialecto, otros tipos y otro comportamiento de generación de valores.

Un experimento pedagógico eficaz define un modelo mínimo Parent/Child y trata de insertar un hijo cuyo `ParentId` no existe. El resultado ilustrativo será distinto según el proveedor:

Proveedor	¿Qué puede demostrar el experimento?
EF Core InMemory	Que el modelo puede almacenarse sin que el proveedor aplique una FK relacional.
SQLite con <code>foreign_keys</code> activo	Que ese proveedor relacional rechaza una FK inexistente.
SQL Server LocalDB	El comportamiento del motor objetivo de <code>AceriaData</code> .

La lección es **no comparar proveedores como si fueran equivalentes**. `SQLite` puede servir para ciertas operaciones relacionales, pero su soporte de BLOB y configuración de concurrencia no reproduce automáticamente el `rowversion` autogenerado por SQL Server.

Tercera capa: integración real con migraciones

Las pruebas de integración de `AceriaData` deben crear una base aislada para la ejecución, apuntar al ensamblado de migraciones correcto y aplicar la **cadena oficial** mediante `MigrateAsync()`. Utilizar `EnsureCreated()` para montar un esquema alternativo haría que la prueba dejase de comprobar la historia real de migraciones.

```

public async Task InitializeAsync()
{
    await using var db = new AceriaDbContext(CreateOptions());
    await db.Database.MigrateAsync();

    await using var connection = new SqlConnection(ConnectionString);
    await connection.OpenAsync();

    _respawner = await Respawner.CreateAsync(connection,
        new RespawnerOptions
        {
            DbAdapter = DbAdapter.SqlServer,
            TablesToIgnore =
                [new Respawn.Graph.Table("__EFMigrationsHistory")]
        });
}

```

El fragmento corresponde al patrón de preparación de un **fixture**, cuyos campos, opciones y gestión de base deben existir en la clase completa. Su papel es preciso: aplicar migraciones antes de construir el limpiador y preservar `__EFMigrationsHistory`. No presenta Respawn como si creara el esquema; limpia **datos de negocio** entre pruebas.

La base de pruebas no debe ser la del alumno: se crea con un identificador específico y se elimina mediante una política de limpieza controlada. Conviene impedir que dos ejecuciones paralelas usen por accidente el mismo nombre.

Prueba que sí exige rowversion real

Una prueba de concurrencia sobre SQL Server debe preparar una orden, crear dos contextos independientes y cargar ambos antes de guardar. El primero modifica y confirma, haciendo avanzar RowVersion; el segundo intenta escribir con el token obsoleto. La **aserción** debe esperar `DbUpdateConcurrencyException` en el segundo guardado:

```

// Fragmento central dentro de un test que ya ha creado dos DbContext
// independientes y una orden con token de concurrencia.
var desdeA = await dbA.OrdenesFabricacion.SingleAsync(o => o.Id == id);
var desdeB = await dbB.OrdenesFabricacion.SingleAsync(o => o.Id == id);

desdeA.Cliente = "Valor A";
await dbA.SaveChangesAsync();

desdeB.Cliente = "Valor B";
await Assert.ThrowsAsync<DbUpdateConcurrencyException>(
    () => dbB.SaveChangesAsync());

```

La comprobación debe acompañarse del estado final desde un tercer contexto. Que la excepción se lance demuestra detección; observar que quedó Valor A confirma qué escritura persistió. El test requiere que el modelo y la base estén realmente configurados con un token de SQL Server, por lo que no se debe trasladar sin adaptación a SQLite.

Cuarta capa: HTTP end-to-end con WebApplicationFactory

Una prueba HTTP no debe limitarse a invocar un método de controlador como si fuera una función ordinaria. `WebApplicationFactory` arranca el host de una aplicación ASP.NET Core y permite enviar peticiones que atraviesan **routing, serialización, dependency injection, lógica de aplicación e infraestructura**.

En `AceriaData`, el escenario combina un endpoint real con una base SQL Server **específica de pruebas**. La conexión de producción o de desarrollo no debe reutilizarse por comodidad. La prueba crea una orden mediante POST, consulta por GET y verifica que el resultado corresponde a la operación. Así se detectan errores de composición que un test unitario de Application no observaría.

```
// Fragmento del escenario real de AceriaData.Api.
var create = await client.PostAsJsonAsync("/api/ordenes", new
{
    NumeroOrden = "OF-HTTP-511",
    Cliente = "Cliente HTTP"
});
Assert.Equal(HttpStatusCode.Created, create.StatusCode);

var response = await client.GetAsync("/api/ordenes/count");
response.EnsureSuccessStatusCode();
var payload = await response.Content.ReadFromJsonAsync<CountResponse>();
Assert.NotNull(payload);
Assert.Equal(1, payload.Total);
```

Las rutas POST /api/ordenes y GET /api/ordenes/count son las de la API real del punto 5.11. El fragmento omite la preparación previa de la fixture, AceriaApiFactory y el tipo auxiliar CountResponse, que se encuentran en el archivo ApiIntegrationTests.cs; no debe copiarse aislado como si fuera un programa completo. El propósito es demostrar la **composición real** del host y SQL Server en una prueba de extremo a extremo. El resultado debe verificarse ejecutando la prueba en su entorno de integración.

Independencia, limpieza y confianza

Un test que sólo pasa porque otro test insertó datos anteriormente es frágil. El fixture prepara el esquema; las operaciones de limpieza restauran los datos a una situación conocida; y cada test establece sus precondiciones. Compartir una base de integración puede ahorrar tiempo, pero exige gestionar cuidadosamente paralelismo y estado para evitar interferencias.

Al interpretar un fallo debemos preguntarnos en qué frontera aparece. Si falla un test unitario, puede haber una regla o colaboración incorrecta. Si falla la integración con SQL Server, revisaremos migraciones, columnas, restricciones y transacciones. Si falla sólo HTTP, pueden intervenir rutas, serialización o configuración del host. Esa separación facilita diagnósticos más precisos.

Conclusión del punto. La pirámide de pruebas no se define por dogmas («todo mock» o «todo base real»), sino por **fidelidad al comportamiento que necesitamos demostrar**. Los mocks prueban colaboración; SQLite prueba parte de la semántica relacional; LocalDB prueba peculiaridades de SQL Server; y WebApplicationFactory comprueba composición HTTP.

Contraprueba real de proveedor: clave foránea InMemory frente a SQLite

En ProviderBehaviorTests.cs el proyecto define un modelo mínimo ProbeParent/ProbeChild y demuestra dos comportamientos opuestos: el proveedor InMemory puede aceptar un hijo con ParentId sin padre y SQLite, con la FK activada y el esquema correspondiente, rechaza esa inserción. No hay que interpretar esto como que un test esté mal escrito: cada motor implementa un contrato distinto.

```
// Variante InMemory: no hay motor relacional que haga cumplir la FK.
var inMemory = new DbContextOptionsBuilder<ProbeContext>()
    .UseInMemoryDatabase(Guid.NewGuid().ToString("N")).Options;
await using var dbMemoria = new ProbeContext(inMemory);
dbMemoria.Children.Add(new ProbeChild { ParentId = 999, Name = "huerfano" });
var error = await Record.ExceptionAsync(() => dbMemoria.SaveChangesAsync());
Assert.Null(error);

// Variante SQLite: conexión ABIERTA mientras dure la base en memoria;
// el proyecto crea explícitamente las tablas con FOREIGN KEY y activa
// PRAGMA foreign_keys=ON antes de comprobar el error.
var sqlite = new SqlConnection("Data Source=:memory:");
await sqlite.OpenAsync();
// Aquí se crea el esquema SQL real del modelo ProbeParent/ProbeChild.
var opciones = new DbContextOptionsBuilder<ProbeContext>()
    .UseSqlite(sqlite).Options;
await using var dbRelacional = new ProbeContext(opciones);
dbRelacional.Children.Add(new ProbeChild { ParentId = 999, Name = "huerfano" });
await Assert.ThrowsAsync<DbUpdateException>(
    () => dbRelacional.SaveChangesAsync());
```

Los tipos ProbeContext, ProbeChild y el SQL de creación de tablas pertenecen al archivo real del repositorio; estos son **extractos**, no un programa autónomo. El objetivo es enseñar que tener entidades EF iguales no proporciona garantías de proveedor equivalentes.

Qué demuestra el test real de rowversion

SqlServerIntegrationTests.RowVersionSqlServer_DetectaConflictoRealEntreDosContextos utiliza el SqlServerDatabaseFixture de AceriaData: crea una orden, comprueba que SQL Server generó una versión no vacía, carga la misma fila en dos contextos, confirma la modificación de A y exige DbUpdateConcurrencyException al intentar guardar B. Ese test **no debe sustituirse por SQLite**: la metadata .IsRowVersion() en un BLOB SQLite no reproduce la generación automática del token de SQL Server.

Respawn restablece datos entre tests pero conserva __EFMigrationsHistory. El fixture aplica **migraciones reales** mediante MigrateAsync, no EnsureCreated; la API mínima de 5.11 permite un test HTTP con WebApplicationFactory sobre la base aislada. El resultado de esa prueba incluye un POST /api/ordenes y un GET /api/ordenes/count, con comprobación de HTTP 201 y total esperado. La única manera de afirmar que la suite pasa en un entorno nuevo es **ejecutarla** con .NET 8, SQL Server LocalDB y dependencias restauradas.

Punto 5.12 — Buenas prácticas y anti-patrones de persistencia empresarial

Un anti-patrón debe tener un síntoma, no sólo un nombre

Es fácil escribir una lista con «evitar N+1», «usar AsNoTracking» o «preferir Fluent API». Esa lista no enseña a diagnosticar un sistema y, además, mezcla recomendaciones contextuales con errores reales. Este punto cierra el módulo convirtiendo cada afirmación en una secuencia: **situación inicial → evidencia observable → cambio propuesto → resultado equivalente → nuevo coste o compromiso**.

La condición previa de cualquier refactorización es que las alternativas resuelvan el mismo problema funcional. Una consulta que devuelve menos datos puede parecer más rápida simplemente porque dejó de cumplir un requisito de la pantalla. Primero se define la salida; después se compara la forma de obtenerla.

Caso 1: N+1 consultas al enumerar órdenes

Una pantalla muestra las órdenes pendientes y, para cada una, la cantidad de planchas asociadas. Una implementación ingenua carga todas las órdenes y ejecuta después otra consulta para contar planchas de cada orden. Si aparecen veinte órdenes, podríamos emitir **una consulta inicial más veinte consultas de recuento**. Ese número es ilustrativo del patrón, no una medición de AceriaData; el contador de comandos debe confirmar la cantidad real en una ejecución.

```
// ANTES: una consulta para las órdenes y otra por cada orden.
var ordenes = await db.OrdenesFabricacion
    .AsNoTracking()
    .Where(o => o.Estado == "Pendiente")
    .ToListAsync();

var resumen = new List<(int Id, string NumeroOrden, int TotalPlanchas)>();
foreach (var orden in ordenes)
{
    var cantidad = await db.PlanchasAcero
        .CountAsync(p => p.OrdenId == orden.Id);
    resumen.Add((orden.Id, orden.NumeroOrden, cantidad));
}
```

En AceriaData, PlanchaAcero declara la FK OrdenId y OrdenFabricacion dispone de la navegación Planchas. El ejemplo utiliza esas propiedades reales. La técnica que estamos enseñando es el recuento correlacionado en bucle, no una cifra de rendimiento ya medida.

Cuando una pantalla sólo necesita un número por orden, una **proyección con agregado** puede expresar el resultado completo como una consulta al proveedor:

```
// DESPUÉS: proyectar el resultado que la pantalla necesita.
var resumenProyectado = await db.OrdenesFabricacion
    .AsNoTracking()
    .Where(o => o.Estado == "Pendiente")
    .OrderBy(o => o.Id)
    .Select(o => new
    {
        o.Id,
        o.NumeroOrden,
        TotalPlanchas = o.Planchas.Count()
    })
    .ToListAsync();
```

La navegación Planchas aparece en las entidades del proyecto y la consulta utiliza su nombre real. El proveedor puede traducir el agregado a una subconsulta correlacionada u otra forma SQL. **No debemos inventar esa sentencia**: se estudia mediante ToQueryString y se observa su ejecución mediante logging o interceptor.

¿Y si la pantalla necesita las planchas completas?

Entonces Include puede evitar el patrón N+1 materializando la navegación requerida:

```
var ordenesConPlanchas = await db.OrdenesFabricacion
    .AsNoTracking()
    .Include(o => o.Planchas)
    .Where(o => o.Estado == "Pendiente")
    .ToListAsync();
```

Esta alternativa puede ser correcta cuando realmente necesitamos todas esas entidades, pero introduce **más columnas y materialización** que una proyección de recuentos. La técnica elegida depende del contrato de salida.

Caso 2: over-fetching y estado rastreado innecesario

Imaginemos una lista que sólo presenta número de orden, cliente, estado y fecha. Cargar todas las propiedades de OrdenFabricacion con tracking recupera información que la vista no muestra y mantiene entidades en el ChangeTracker sin que vayan a modificarse.

Implementación inicial:

```
var entidades = await db.OrdenesFabricacion
    .Where(o => o.Estado == "Pendiente")
    .ToListAsync();
```

Proyección adaptada a la pantalla:

```
var resumen = await db.OrdenesFabricacion
    .AsNoTracking()
    .Where(o => o.Estado == "Pendiente")
    .OrderBy(o => o.Id)
    .Select(o => new
    {
        o.NumeroOrden,
        o.Cliente,
        o.Estado,
        o.FechaCreacion
    })
    .ToListAsync();
```

La comparación correcta inspecciona qué columnas aparecen en el SQL de cada consulta y qué entidades quedan en el tracker. Si la entidad incluye RowVersion, cabe comprobar que la primera materializa esa columna mientras la segunda no la necesita para el resumen. Pero sería incorrecto concluir que **todo tracking es malo**: si el caso de uso edita y persiste la entidad, el seguimiento puede ser precisamente el mecanismo apropiado.

Caso 3: evaluación de cliente y traducción SQL

Una consulta LINQ no siempre puede traducirse a SQL. En EF Core 8, utilizar un método .NET arbitrario no traducible dentro del predicado Where normalmente produce una excepción en lugar de traer silenciosamente toda la tabla y filtrar en el cliente. **Atención al contexto del código**: una función local no se puede referenciar desde un árbol de expresión y puede producir CS8110 en compilación; para reproducir el fallo de traducción debe utilizarse un método estático definido en la clase, tal como hace AceriaData.

```
// Dentro de un método del repositorio: la función auxiliar es un
// MÉTODO ESTÁTICO DE LA CLASE, definido fuera de este método.
var consulta = db.OrdenesFabricacion
    .Where(o => EsClienteEspecial(o.Cliente));

// Fuerza traducción/ejecución y permite observar el fallo de EF Core.
try { var filas = consulta.ToList(); }
catch (InvalidOperationException ex) { Console.WriteLine(ex.Message); }
```

```
// En el cuerpo de la clase (NO dentro del método):
// private static bool EsClienteEspecial(string nombre) =>
//     nombre.Trim().StartsWith("AC-", StringComparison.OrdinalIgnoreCase);
```

El diagnóstico distingue **componer la expresión** de **ejecutarla**: la traducción suele producirse al enumerar el resultado, no simplemente al asignar consulta.

Si aceptamos filtrar en memoria, podemos cruzar conscientemente la frontera con `AsEnumerable()` después de una selección servidor adecuada. Pero entonces el coste cambia: la aplicación puede transferir más filas y consumir más memoria.

```
// Cargar explícitamente datos seleccionados antes del filtro local.
```

```
var candidatos = db.OrdenesFabricacion
    .AsNoTracking()
    .Select(o => new { o.Id, o.Cliente })
    .AsEnumerable();

var especiales = candidatos
    .Where(o => EsClienteEspecial(o.Cliente))
    .ToList();
```

Esta versión no debe presentarse como optimización universal. Sólo es razonable si el volumen transferido y las reglas justifican realizar esa parte del trabajo en .NET. En muchos casos puede ser mejor reformular el predicado con funciones que el proveedor sí traduzca.

Caso 4: AsSplitQuery y consultas con múltiples colecciones

Cargar varias colecciones mediante un único SQL con JOIN puede multiplicar filas por combinaciones de relaciones; dividir la carga en consultas separadas puede reducir esa **explosión cartesiana**. Pero `AsSplitQuery` no es automáticamente superior: incrementa los roundtrips y puede observar cambios de datos entre las consultas bajo ciertos niveles de aislamiento.

Para decidir debemos conocer cuántas colecciones se cargan, el tamaño de los resultados, los índices, la latencia y el requisito de consistencia temporal. El comparativo debe verificar equivalencia funcional y observar comandos SQL reales. Sin estas condiciones, convertir «usar SplitQuery» en regla universal conduce a decisiones mecánicas equivocadas.

Caso 5: configuración, sargabilidad y abstracciones

Data Annotations frente a Fluent API. Ambas son opciones válidas de configuración. Fluent API permite expresar ciertos modelos más complejos y centralizar las decisiones de mapeo en Infrastructure. Data Annotations pueden ser suficientes para restricciones sencillas. No debe etiquetarse ninguna de las dos como anti-patrón por su mera existencia.

Sargabilidad. Aplicar una función sobre una columna dentro de un predicado puede dificultar el aprovechamiento de un índice, pero la consecuencia depende de la expresión concreta, proveedor, collation y diseño físico. La respuesta profesional consiste en inspeccionar el plan de ejecución, no afirmar que «cualquier función inutiliza todos los índices».

Repository. Añadir una capa por obligación puede multiplicar abstracciones sin valor. Sin embargo, un repositorio específico puede delimitar claramente Application e Infrastructure. El criterio es si resuelve una necesidad arquitectónica, no si aparece o no en una lista de buenas prácticas genéricas.

Ciclo de vida de DbContext. Mantener un contexto vivo demasiado tiempo puede acumular entidades rastreadas y exponer estado obsoleto. La alternativa suele ser definir unidades de trabajo cortas y claras. Tampoco significa que cada consulta tenga que abrir una conexión física nueva; hablamos de límites de contexto, scopes, pooling y gestión de recursos según el escenario.

Matriz final: síntoma, demostración y decisión

Situación	Evidencia principal	Cambio a estudiar	Compromiso
N+1	Cantidad de comandos ejecutados frente a cantidad de órdenes.	Proyección o carga de relaciones.	Forma de salida y materialización.
Over-fetching	Columnas seleccionadas y entradas en tracker.	Proyectar a DTO.	No disponer de la entidad para edición inmediata.
Método no traducible	Excepción de traducción en ejecución.	Reformular para SQL o filtrar conscientemente en cliente.	Transferencia y memoria.
Múltiples colecciones	Forma del SQL y multiplicación de filas.	Comparar consulta única con split query.	Roundtrips y consistencia.
Contexto de larga vida	Estado rastreado y datos obsoletos.	Reducir duración de unidad de trabajo.	Gestión de ciclos de vida.
Migración divergente	Snapshot, historial y esquema no alineados.	Regeneración privada o migración correctiva.	Coordinación del equipo.
Logging excesivo	Volumen y datos registrados.	Filtrar, rotar y correlacionar.	Detalle disponible durante diagnósticos.

Cierre del módulo: un método común

Los doce puntos pueden entenderse como una única disciplina de ingeniería. Para un problema de persistencia, primero definimos **qué resultado significa que la aplicación funciona correctamente**. Después identificamos la frontera involucrada: contexto, transacción, versión de fila, esquema, capa arquitectónica o proveedor de pruebas. Buscamos evidencia que pueda confirmar el problema, elegimos una alternativa proporcionada y comprobamos que conserva las invariantes importantes.

En concurrencia, la evidencia incluye valores originales, modificaciones de los actores, SQL y estado final. En transacciones, importa qué cambios quedaron confirmados tras el fallo. En migraciones, debemos comparar historial y esquema. En optimización, primero probamos equivalencia funcional y después medimos comandos, columnas, tracking y rendimiento bajo una carga representativa. En observabilidad, no basta con producir señales: necesitamos saber qué representan y dónde se recibieron.

El aprendizaje central no es memorizar APIs, sino razonar sobre los límites y la evidencia de una operación persistente. Esa capacidad permite llevar AceriaData, y después cualquier aplicación empresarial, desde ejemplos correctos en un entorno local hasta sistemas que pueden revisarse, desplegarse, diagnosticarse y mantenerse de manera responsable.

Comparar realmente las tres implementaciones de N+1

El código de `BuenasPracticasAntiPatronesM5Diagnostico.cs` (Infraestructure, punto 5.12) ya hace una comparación que el capítulo debe explicar: carga cabeceras, cuenta planchas mediante una consulta por orden, repite mediante `Include` (o `=> o.Planchas`) y luego mediante una proyección con `o.Planchas.Count`. Antes de cada variante limpia el tracker y reinicia `SqlCommandCounterInterceptor.Instance`. Después recoge conteos, SQL de las variantes y resultados.

// Equivalencia funcional ANTES de interpretar roundtrips.

```
var equivalentes =
    cabeceras.Count == conInclude.Count &&
    conInclude.Count == proyectadas.Count &&
    planchasNMasUno == planchasInclude &&
    planchasInclude == planchasProyeccion;
if (!equivalentes)
    throw new InvalidOperationException("Las versiones devuelven datos distintos.");
```

```
Console.WriteLine($"N+1: {consultasNMasUno} comandos");
Console.WriteLine($"Include: {consultasInclude} comandos");
Console.WriteLine($"Proyección: {consultasProyeccion} comandos");
```

Las variables proceden de las tres mediciones del diagnóstico, no se presentan como método compilable por sí solo. **Sin verificar equivalencia, una menor cantidad de consultas podría deberse simplemente a que una alternativa devuelve menos información.** El test de integración `BuenasPracticasAntiPatronesM5Tests` protege estos invariantes de la refactorización.

Distinguir tres clases de evidencia

1. **ToQueryString()** inspecciona el SQL que EF Core prepararía para una consulta LINQ; no prueba que el comando se haya ejecutado.
2. **Interceptor y logging** muestran comandos que alcanzaron la infraestructura de base, y permiten contar consultas reales en el escenario concreto.
3. **Datos finales y ChangeTracker** permiten comprobar equivalencia y rastreo: una proyección no debe compararse sólo con el conteo de comandos, sino también con columnas transferidas y objetos materializados.

La demostración del método no traducible se corresponde con `BuenasPracticasAntiPatronesM5Diagnostico.MedirTrac` donde `EsPendiente` es un **método estático de clase**. Si el manual declara la función dentro de otro método, es posible que el compilador rechace el árbol de expresión antes de ejecutar EF Core; debe evitarse esa construcción.

Glosario breve

Actualización perdida. Escritura que reemplaza un cambio anterior confirmado sin que el segundo actor haya tenido conocimiento de la edición intermedia.

Token de concurrencia. Propiedad cuyo valor original interviene en la condición de escritura para detectar que los datos leídos han cambiado.

rowversion. Tipo binario de SQL Server actualizado por el motor al modificar una fila; no es una marca horaria ni crea automáticamente un índice.

OriginalValues / CurrentValues. Valores de referencia y valores que EF Core mantiene para una entidad rastreada y su intención de persistencia.

Savepoint. Punto de recuperación dentro de una transacción que permite revertir parcialmente operaciones posteriores.

Transacción ambiental. Transacción que los recursos compatibles pueden descubrir mediante `Transaction.Current` durante un `TransactionScope`.

Promoción distribuida. Situación en la que la coordinación de recursos requiere pasar de una transacción local a una infraestructura de transacciones distribuidas.

Migration bundle. Artefacto ejecutable de EF Core para aplicar migraciones sin instalar la herramienta `dotnet ef` en el destino.

Script idempotente. Script de migraciones con comprobaciones que evitan volver a aplicar migraciones ya registradas, para estados válidos de una cadena determinada.

Snapshot. Representación del modelo utilizada por EF Core para determinar cómo evolucionar la cadena de migraciones.

Drift. Diferencia no prevista entre el historial de migraciones, el modelo esperado y el esquema físico.

Respawn. Herramienta utilizada para limpiar datos entre tests de integración preservando el esquema y, cuando se configura, el historial de migraciones.

WebApplicationFactory. Infraestructura de pruebas de ASP.NET Core que inicia un host y permite validar peticiones HTTP atravesando las dependencias reales configuradas.

DiagnosticListener. Mecanismo de publicación/suscripción de eventos de diagnóstico en el proceso.

EventCounters. Contadores publicados por `EventSource` que permiten observar métricas agregadas del proceso.

N+1. Patrón en el que una consulta inicial va seguida de una consulta por cada elemento principal, cuando podría utilizarse otra estrategia de acceso a datos.

Over-fetching. Recuperación o materialización de datos que el consumidor no necesita.

Sargabilidad. Propiedad de un predicado que, bajo ciertas condiciones del motor y los índices, permite accesos eficientes mediante estructuras de búsqueda.

Equivalencia funcional. Condición que exige a dos implementaciones comparadas entregar el mismo resultado de negocio requerido antes de comparar rendimiento o coste.